Power Electronics I

Course Project Report

Semi-Controlled Rectifiers for Battery Charging

Team Members

Mohammed Azab

Basant Saber

Habiba Abdelhafez

German International University in Berlin
Faculty of Engineering

Instructor: Dr.-Ing. Moustafa Adly

GitHub Repository:

https://github.com/Mohammed-Azab/thyristolab

Contents

Abstract

This project presents the design and analysis of controlled rectifier circuits using thyristors (SCRs) for battery charging applications. Three different rectifier configurations are implemented and simulated in MATLAB: half-wave controlled rectifier, full-wave center-tapped rectifier, and full-wave bridge rectifier. The study examines the relationship between firing angle and charging characteristics, including average and RMS voltage and current values. Performance comparisons demonstrate that full-wave configurations provide superior charging performance with reduced charging times and improved efficiency compared to half-wave rectifiers. The analysis includes circuit modeling, waveform generation, and comprehensive performance metrics for each topology.

1 Introduction

1.1 Objectives

The aim of this project is to design and simulate controlled rectifiers using thyristors (SCRs) for battery charging applications. A rectifier converts AC signals into DC signals by blocking the negative half of the AC waveform, making the output unidirectional for proper battery charging.

Three rectifier configurations were developed in MATLAB:

1. **Half-wave Controlled Rectifier**: Single thyristor allowing only positive half-cycles
2. **Full-wave Center-Tapped Rectifier**: Center-tapped transformer with two thyristors
3. **Full-wave Bridge Rectifier**: Four thyristors in bridge configuration

The analysis examines the relationship between firing angle and charging characteristics (average and RMS voltage, current, charging time) to compare the performance of each configuration.

1.2 Background

For safe and efficient operation, batteries require stable DC current with controlled voltage and current. Silicon-Controlled Rectifiers (SCRs) regulate power precisely, preventing overcharging and ensuring long battery life. SCRs can handle high currents and powers while adapting to battery state of charge (SoC) and temperature.

1.3 Scope

This project covers:

- Analytical modeling using MATLAB
- Three rectifier configurations
- Performance comparison based on firing angle
- Power loss analysis including thyristor non-idealities
- State of Charge tracking

2 Design and Implementation

2.1 MATLAB Function Architecture

Each rectifier type is implemented as a separate MATLAB function:

- `half_wave_charger.m` - Half-wave controlled rectifier
- `full_wave_ct_charger.m` - Full-wave center-tapped rectifier
- `full_wave_bridge_charger.m` - Full-wave bridge rectifier
- `params.m` - System parameter configuration

All functions share common features:

1. Input parameter handling for battery, supply, and thyristor specifications
2. Firing angle sweep from 0° to 180°
3. Calculation of voltage, current, and charging metrics
4. Power loss analysis (battery, thyristor conduction, blocking, switching)
5. State of Charge tracking
6. Comprehensive visualization and data output

2.2 Half-Wave Controlled Rectifier

Uses a single thyristor that conducts only during positive half-cycles when:

- Thyristor is forward-biased
- Gate trigger is applied ($\theta \geq \alpha$)
- Output voltage exceeds battery EMF

Key Equations:

$$V_{dc} = \frac{V_m}{2\pi}(1 + \cos\alpha) - V_t \tag{1}$$

$$I_{avg} = \frac{V_{dc} - V_{bat}}{R_{bat}} \tag{2}$$

$$t_{charge} = \frac{(SoC_{target} - SoC_{init}) \cdot C_{Ah} \cdot 3600}{I_{avg}} \tag{3}$$

2.3 Full-Wave Center-Tapped Rectifier

Employs center-tapped transformer with two thyristors alternating each half-cycle, producing two output pulses per AC cycle.

Advantages: Higher average voltage, reduced ripple, faster charging

Key Equation:

$$V_{dc} = \frac{2V_m}{\pi}(1 + \cos\alpha) - V_t \tag{4}$$

2.4 Full-Wave Bridge Rectifier

Four thyristors in bridge arrangement. T1-T3 conduct during positive half-cycle, T2-T4 during negative half-cycle.

Advantages: Standard transformer (no center-tap), better transformer utilization
Trade-off: Four thyristors vs two thyristor drops in series
Key Equation:

$$V_{dc} = \frac{2V_m}{\pi}(1 + \cos\alpha) - 2V_t \tag{5}$$

2.5 Power Loss Modeling

Battery internal losses:

$$P_{batt} = I_{rms}^2 \cdot R_{bat} \tag{6}$$

Thyristor conduction losses:

$$P_{th,cond} = V_t \cdot I_{avg} + R_{th} \cdot I_{rms}^2 \tag{7}$$

Thyristor blocking losses:

$$P_{block} = V_{block,avg} \cdot I_{leak} \tag{8}$$

Switching losses:

$$P_{switch} = f \cdot (E_{on} + E_{off}) \tag{9}$$

where $E_{on} = \frac{1}{6}V_{block}I_{peak}t_{rise}$ and $E_{off} = \frac{1}{6}V_{block}I_{peak}t_{fall}$.

2.6 State of Charge Tracking

$$SoC(t) = SoC_0 + \frac{100}{Q_{capacity}} \int_0^t I_{charge}(\tau)d\tau \tag{10}$$

where $Q_{capacity}$ is in Coulombs (Ah $\times$ 3600).

3 Results and Analysis

3.1 System Parameters

Table 1: System Parameters

Parameter	Value
Supply Voltage (RMS)	230 V
Supply Frequency	50 Hz
Battery Voltage	20 V
Internal Resistance	0.1 Ω
Battery Capacity	50 Ah
Initial SoC	12 %
Target SoC	100 %
Thyristor Forward Drop	1.5 V

3.2 Half-Wave Rectifier Results

Figure 1 shows voltage waveforms at $\alpha = 30°$. Output consists of controlled positive half-cycles with conduction from firing angle to end of positive cycle.

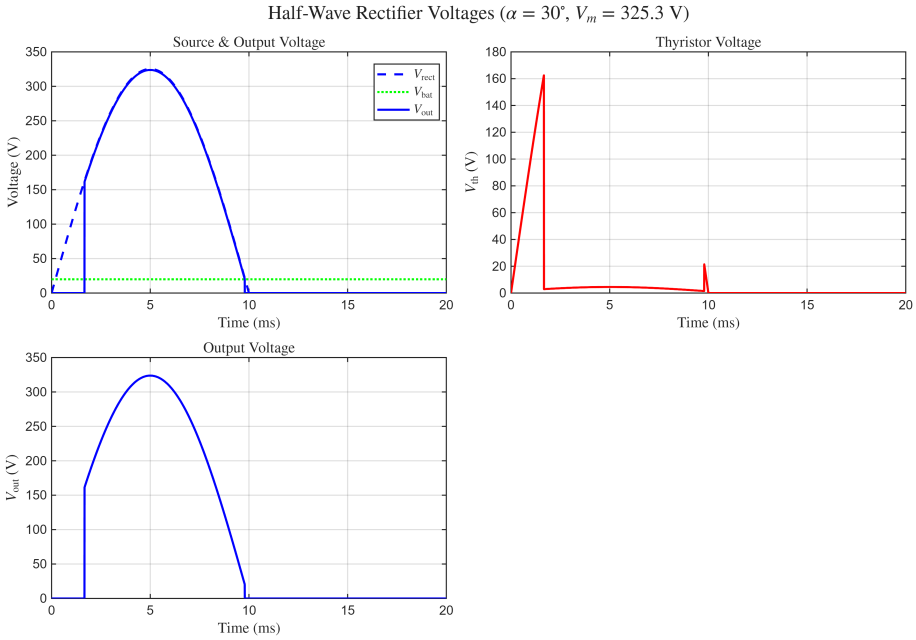

Figure 1: Half-wave rectifier voltages at $\alpha = 30°$

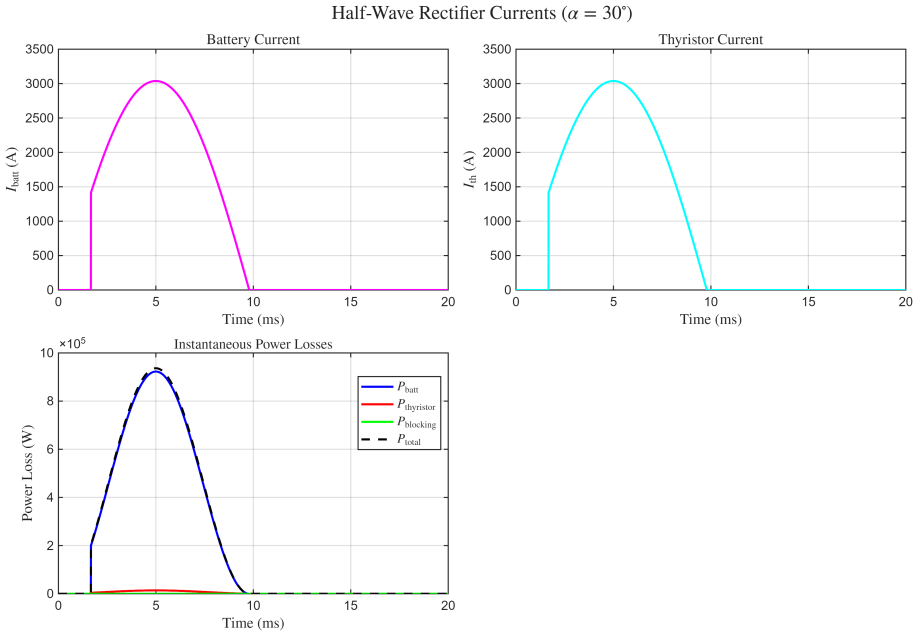

Figure 2: Half-wave rectifier currents at $\alpha = 30°$

Figure 3 shows charging time increases exponentially with firing angle due to reduced average voltage and current.

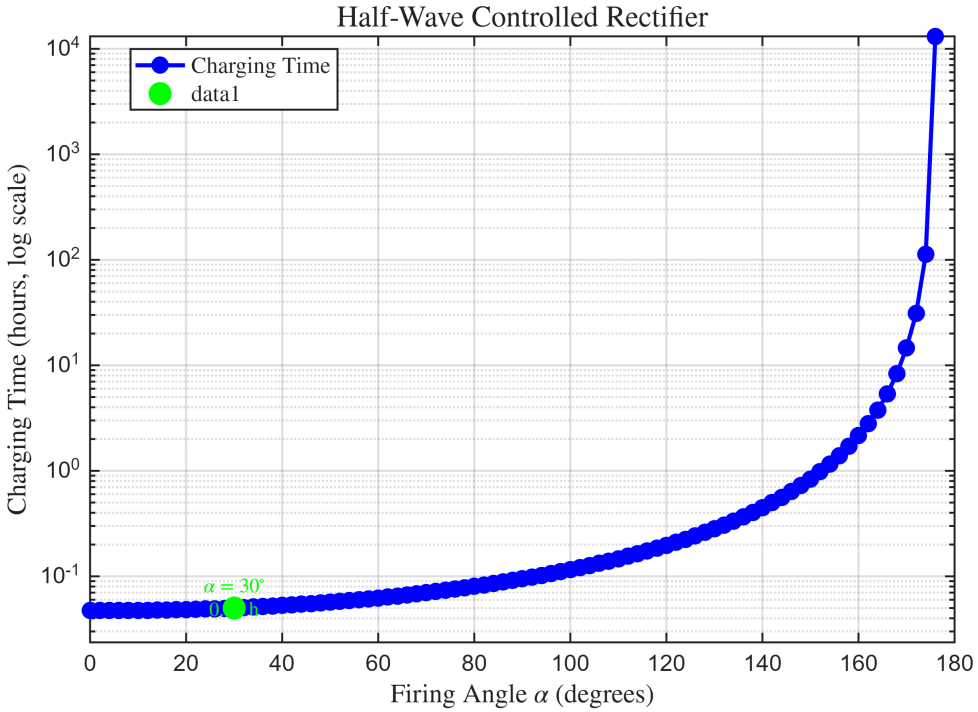

Figure 3: Charging time vs firing angle (half-wave)

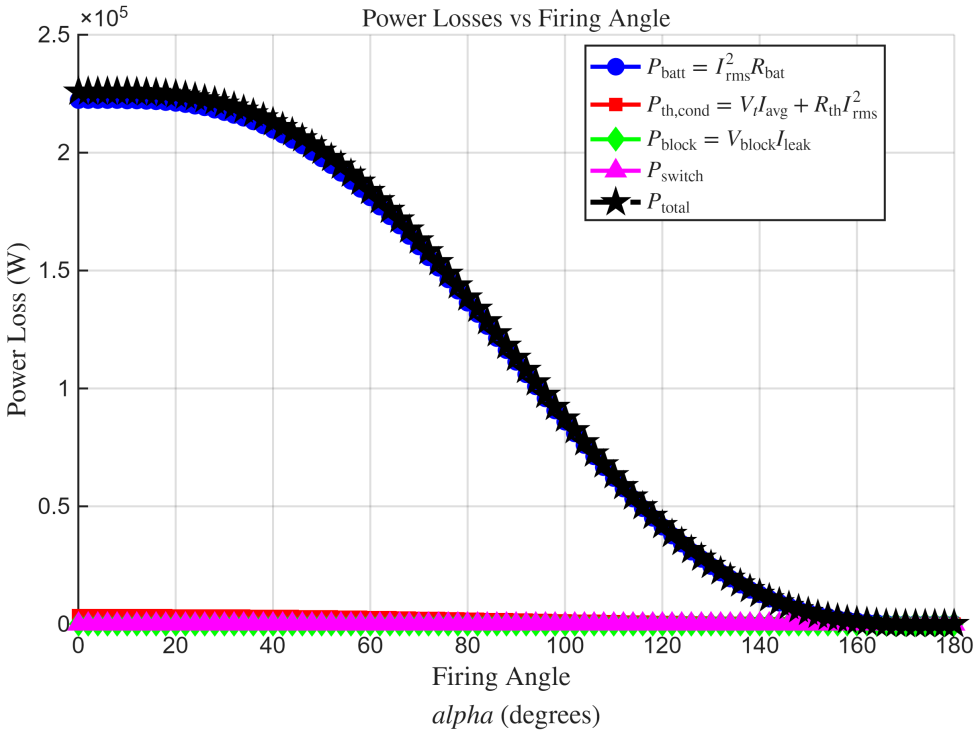

Figure 4: Power losses vs firing angle (half-wave)

3.3 Full-Wave Center-Tapped Rectifier Results

Full-wave configuration produces two output pulses per AC cycle, significantly improving performance.

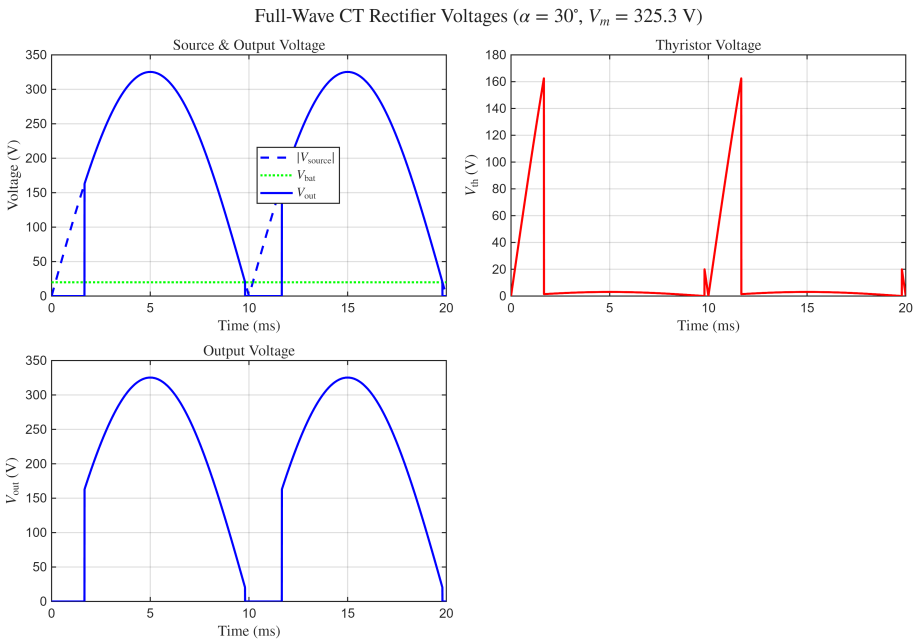

Figure 5: Full-wave CT voltages at $\alpha = 30°$

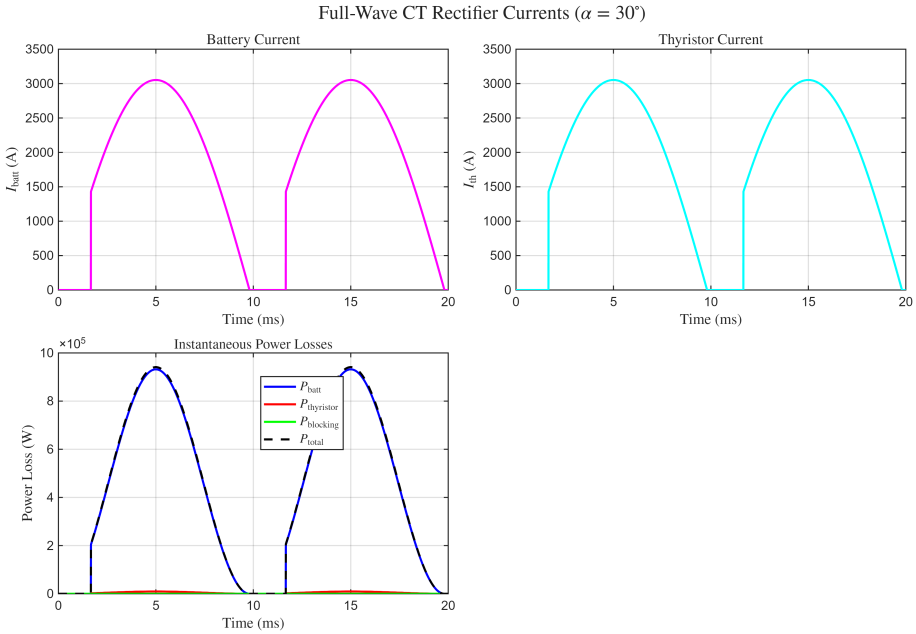

Figure 6: Full-wave CT currents at $\alpha = 30°$

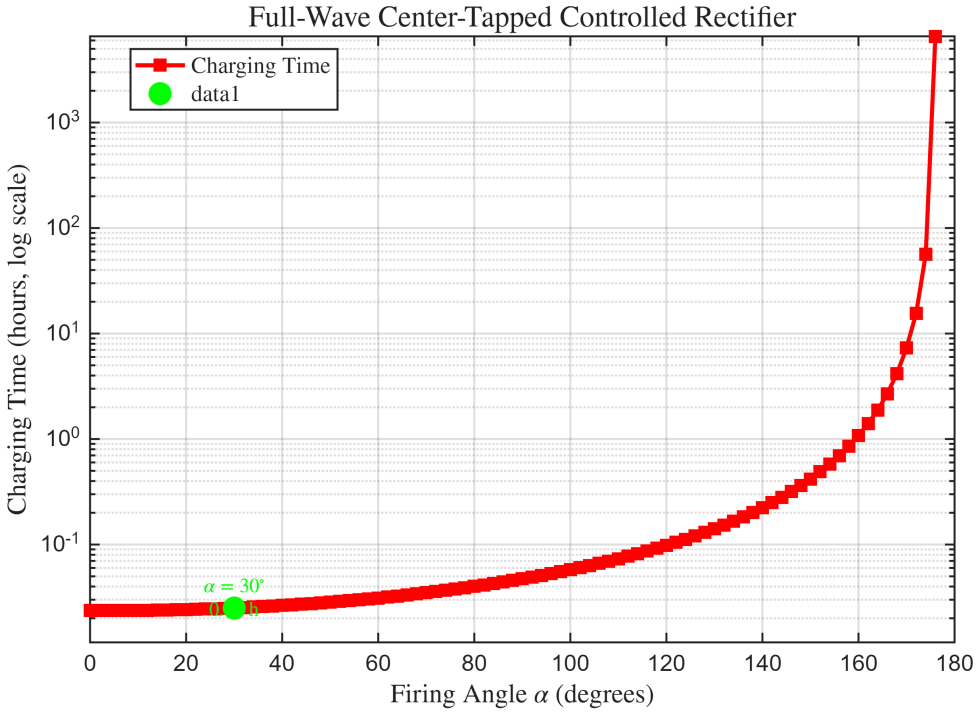

Figure 7: Charging time vs firing angle (full-wave CT)

Full-wave achieves approximately 50% faster charging compared to half-wave for all firing angles.

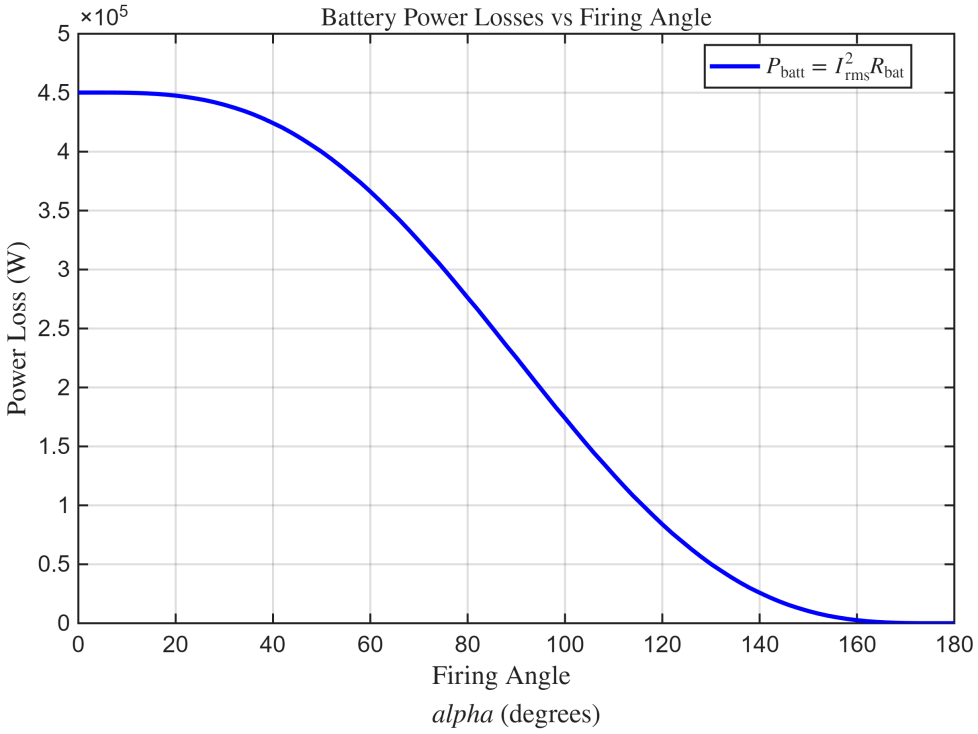

Figure 8: Power losses vs firing angle (full-wave CT)

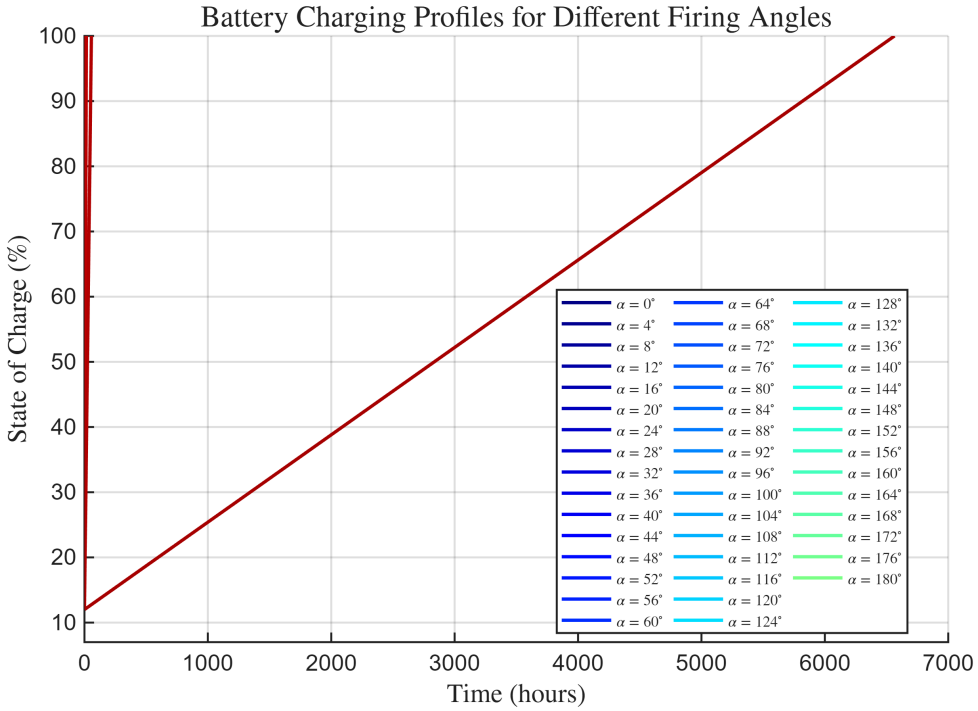

Figure 9: SoC progression for different firing angles (target SoC scenario)

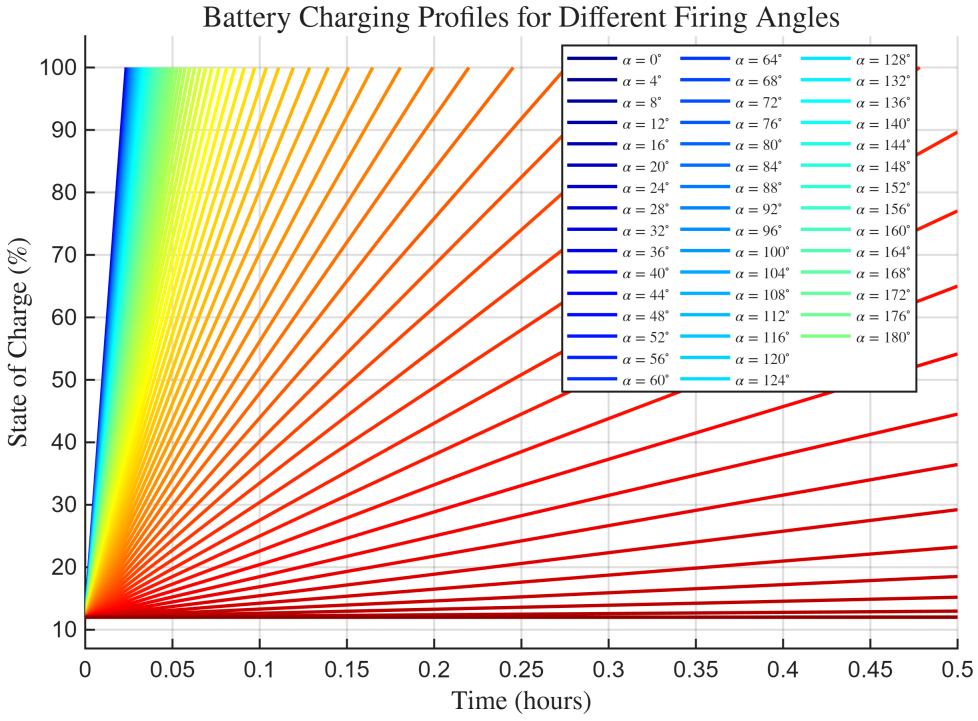

Figure 10: SoC progression for different firing angles (fixed charging duration)

3.4 Half-Wave SoC Analysis

Figure 11 shows battery state of charge progression over time for the half-wave rectifier with fixed charging duration. Different firing angles result in varying final SoC levels, with smaller angles achieving higher charge levels in the same time period.

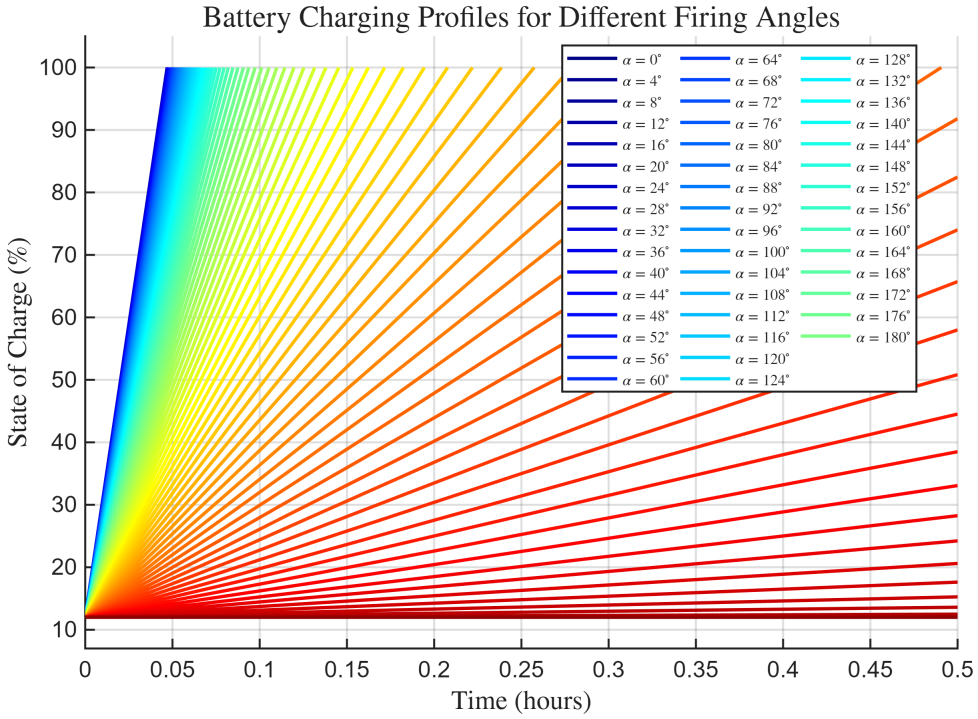

Figure 11: Half-wave SoC vs time (fixed charging duration)

3.5 Full-Wave Bridge Rectifier Results

The bridge configuration uses four thyristors and provides performance similar to the center-tapped design but with a standard (non-center-tapped) transformer.

3.5.1 Charging Time Analysis

Figure 12 illustrates how charging time varies with firing angle for the bridge rectifier. The relationship is similar to other configurations, with charging time increasing exponentially as the firing angle approaches 90° due to reduced average output voltage and current.

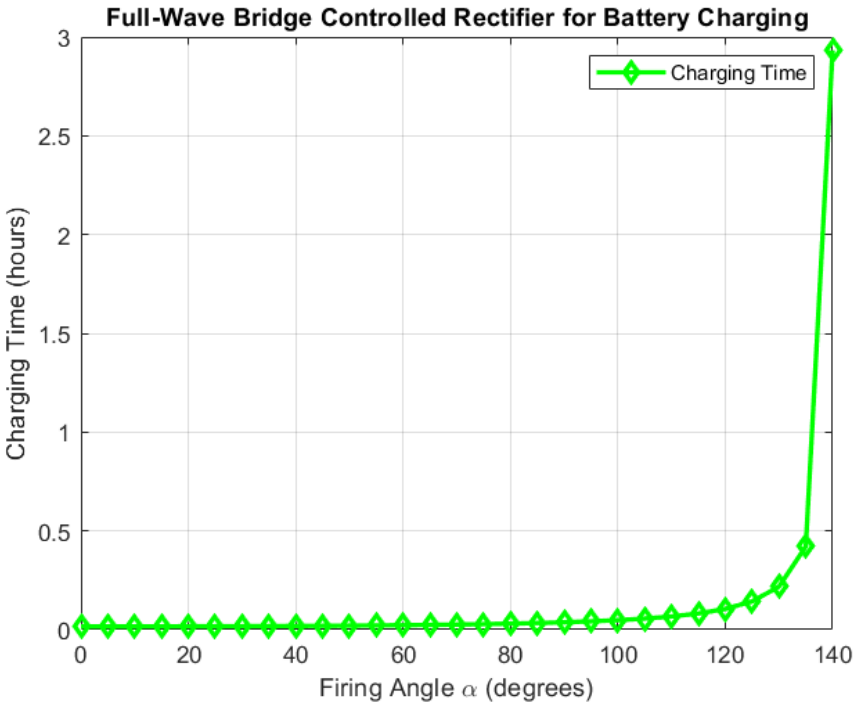

Figure 12: Bridge rectifier charging time vs firing angle

3.5.2 Power Loss Analysis

Figure 13 shows power losses as a function of firing angle. The bridge configuration exhibits two thyristor drops per conduction path (compared to one in center-tapped), resulting in slightly higher conduction losses. Power losses generally decrease with increasing firing angle due to reduced current flow.

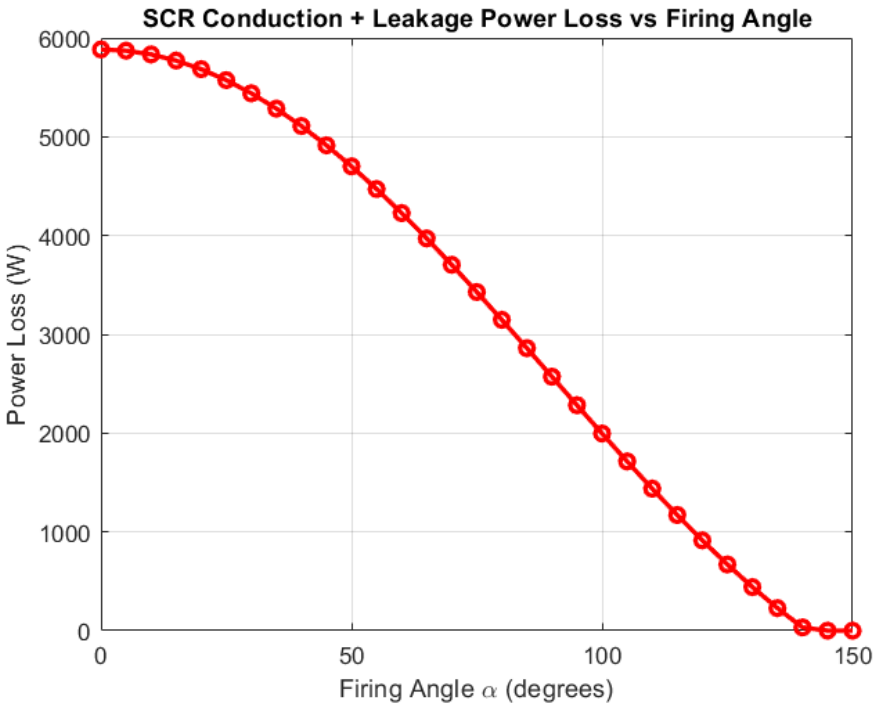

Figure 13: Bridge rectifier power losses vs firing angle

3.6 Comparative Analysis

3.6.1 Performance Comparison

1. **Half-Wave**: Simplest (one thyristor), slowest charging, highest ripple
2. **Full-Wave CT**: Two thyristors + center-tapped transformer, 50% faster charging, reduced ripple
3. **Full-Wave Bridge**: Four thyristors + standard transformer, similar performance to CT, no center-tap required

3.6.2 Key Observations

- Firing angle provides flexible control of charging current
- Low firing angles ($\alpha < 60°$) enable fast charging but higher losses
- Full-wave configurations significantly superior for practical applications
- Thyristor forward drop has measurable impact on performance
- SoC tracking enables intelligent charging strategies

4 Discussion

4.1 Analysis of Results

4.1.1 Firing Angle Effect

Increasing firing angle significantly increases charging time because:

- Delays thyristor turn-on, reducing conduction period
- Average voltage decreases: $V_{dc} = \frac{V_m}{k\pi}(1 + \cos\alpha)$ where $k = 2$ (half-wave), $k = 1$ (full-wave)
- Lower voltage reduces charging current: $I_{avg} = (V_{dc} - V_{bat})/R_{bat}$
- Relationship is highly nonlinear with exponential growth as $\alpha \to 180$

Practical firing angles: 0° to 90°. Beyond 90°, output voltage too low for effective charging.

4.1.2 Half-Wave vs Full-Wave

Full-wave rectifiers are clearly superior:

- 50% reduction in charging time (two pulses per cycle)
- Double ripple frequency (100Hz vs 50Hz) - easier filtering
- Better transformer utilization
- Lower harmonic content

Half-wave only justified for very low-power, cost-sensitive applications.

4.1.3 Center-Tapped vs Bridge

Center-Tapped:

- Pros: 2 thyristors, one V_t drop, simpler gate drive
- Cons: Requires center-tapped transformer, poor transformer utilization

Bridge:

- Pros: Standard transformer, better utilization, widely available
- Cons: 4 thyristors, two V_t drops, complex gate drive

Selection: Use bridge for most applications (standard transformers). Use center-tapped if already available or when minimizing voltage drops is critical.

5 Load Analysis and Rectifier Performance Study

5.1 Objectives

This section presents a comprehensive analysis of controlled rectifier circuits operating with different load characteristics. The primary objectives of this study are:

1. **Simulate half-wave and full-wave controlled rectifiers** using thyristors (SCRs) with varying firing angles to understand phase-controlled rectification behavior
2. **Study the effect of different load types** including purely resistive (R), resistive-inductive (RL), and highly inductive loads on current and voltage waveforms
3. **Analyze the influence of firing angle** (α) on average and RMS output voltage and current across the full control range (0° to 180°)
4. **Determine the role of free-wheeling diodes** in improving rectifier performance, particularly for inductive loads
5. **Compare the performance** of half-wave, center-tapped full-wave, and bridge full-wave rectifier configurations under various operating conditions
6. **Investigate special operating modes** including current continuity for inductive loads and cases of positive load current with negative output voltage
7. **Design firing circuits** using pulse generators with adjustable phase delay for thyristor gate control

5.2 System Specifications

The controlled rectifier systems analyzed in this study operate under the following specifications:

- **Supply Voltage:** 230 V RMS, 50 Hz single-phase sinusoidal source
- **Load Configurations:**
  - Resistive only: $R = 10\ \Omega$, $L \approx 0$ H
  - R-L load: $R = 10\ \Omega$, $L = 50$ mH
  - Highly inductive: $R = 5\ \Omega$, $L = 200$ mH
- **Firing Angle Range:** Adjustable from 0° to 180°
- **Rectifier Topologies:** Half-wave, center-tapped full-wave, and bridge full-wave
- **Free-wheeling Diode:** Analyzed both with and without FWD for each configuration

5.3 Measured Parameters

For each rectifier configuration and load type, the following parameters are measured and analyzed:

- Load voltage and current waveforms (instantaneous values)
- Thyristor voltage and current waveforms
- Average load voltage as a function of firing angle
- RMS load voltage and current
- Current conduction angles and continuity
- Power factor and harmonic content

6 Methodology and Simulation Setup

6.1 Simulation Environment

The load analysis simulations were conducted using MATLAB/Simulink with Simscape Electrical components. This environment provides accurate modeling of power electronic components including thyristors, diodes, transformers, and passive load elements. The simulation approach combines:

- **Simscape circuit models** for accurate physical behavior of semiconductor devices
- **MATLAB scripting** for automated parameter sweeps and data analysis
- **Pulse generators** for precise thyristor gate signal timing and firing angle control

6.2 Firing Circuit Design

The thyristor firing circuits are implemented using Simulink Pulse Generator blocks with the following design parameters:

- **Gate Signal Amplitude:** 10 V (sufficient to trigger thyristor)
- **Pulse Width:** 50% of line period (ensuring reliable triggering)
- **Period:** 20 ms (synchronized to 50 Hz AC supply)
- **Phase Delay:** Calculated as $t_d = \frac{\alpha}{360} \times T$ where $T = 1/f$

For full-wave rectifiers, two pulse generators are used with appropriate phase relationships:

- **Center-tapped:** Second pulse delayed by $T/2$ (10 ms)
- **Bridge:** Thyristor pairs triggered with π phase shift

6.3 Automated Sweep Analysis

The `load_analysis_sweep.m` MATLAB script automates the simulation process across multiple firing angles and load scenarios. The script performs the following operations:

1. **Model Selection:** Loads specified Simulink models (half-wave, center-tapped, or bridge)
2. **Parameter Configuration:** Iterates through firing angles (0° to 180° in 5° steps) and load scenarios
3. **Simulation Execution:** Runs each configuration for 0.1 seconds (5 complete AC cycles)
4. **Data Extraction:** Captures voltage and current waveforms from To Workspace blocks
5. **Metric Calculation:** Computes V_{avg}, V_{rms}, I_{avg}, I_{rms} for each case
6. **Result Storage:** Saves results to MAT files for post-processing
7. **Graph Generation:** Automatically generates and saves waveform plots for selected α values

This automated approach ensures consistent and repeatable analysis across all rectifier configurations and operating conditions.

6.4 Load Scenario Definitions

Three distinct load scenarios are analyzed to understand rectifier behavior across a spectrum of load characteristics:

6.4.1 Resistive Only Load

- $R = 10\ \Omega$, $L = 1\ \mu$H (minimal inductance for numerical stability)
- Represents pure resistive loads such as heating elements
- Current instantaneously follows voltage
- No energy storage; discontinuous current at high firing angles

6.4.2 Resistive-Inductive (R-L) Load

- $R = 10\ \Omega$, $L = 50$ mH
- Time constant: $\tau = L/R = 5$ ms (comparable to line period)
- Represents typical motor loads or inductive heating
- Current lags voltage; smoother waveforms
- Extended conduction beyond voltage zero-crossings

6.4.3 Highly Inductive Load

- $R = 5\ \Omega$, $L = 200$ mH
- Time constant: $\tau = L/R = 40$ ms (much larger than line period)
- Represents DC motor armatures, large inductors
- Nearly constant current; continuous conduction
- Enables four-quadrant operation and regenerative braking

7 Simulation Results

7.1 Half-Wave Controlled Rectifier

The half-wave rectifier uses a single thyristor to control current flow during the positive half-cycle of the AC supply.

7.1.1 Resistive Load

Without Free-Wheeling Diode:

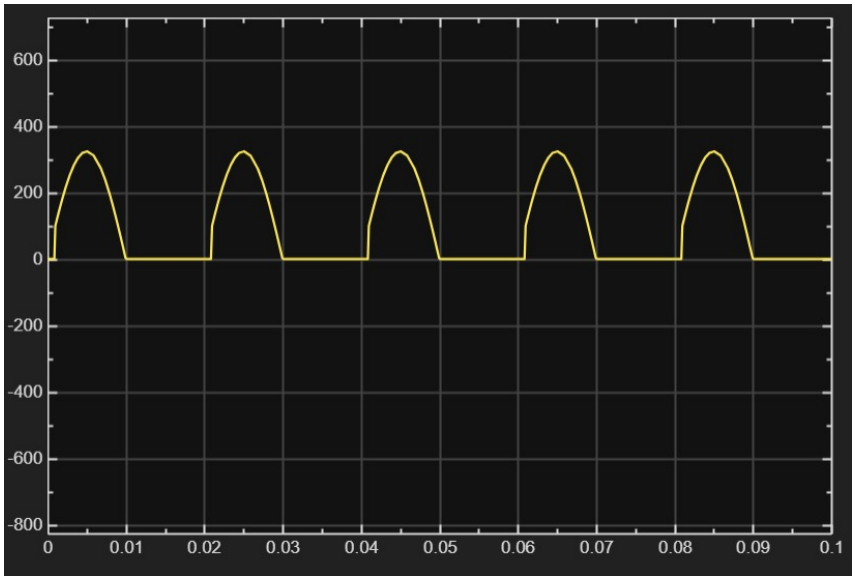

Figure 14: Half-wave rectifier output voltage for resistive load without FWD. Discontinuous voltage pulses during positive half-cycles only.

Without Free-Wheeling Diode:

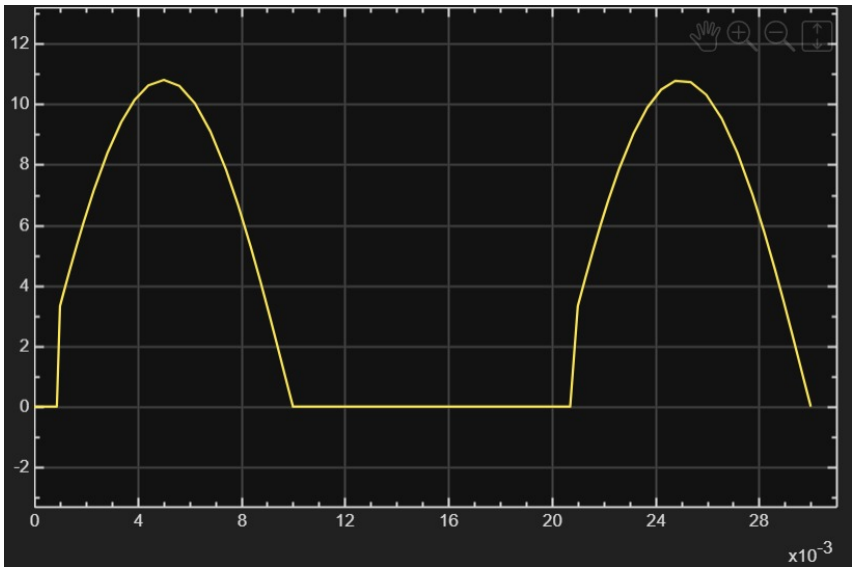

Figure 15: Half-wave rectifier output current for resistive load without FWD. Discontinuous current pulses during positive half-cycles only.

With Free-Wheeling Diode:

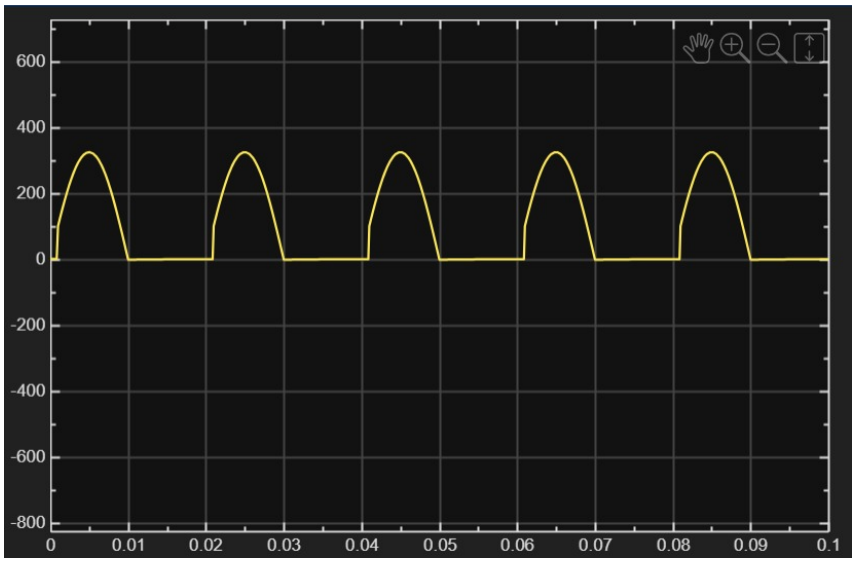

Figure 16: Half-wave rectifier output voltage for resistive load with FWD. For resistive loads, the FWD has minimal effect since current naturally goes to zero when voltage is zero.

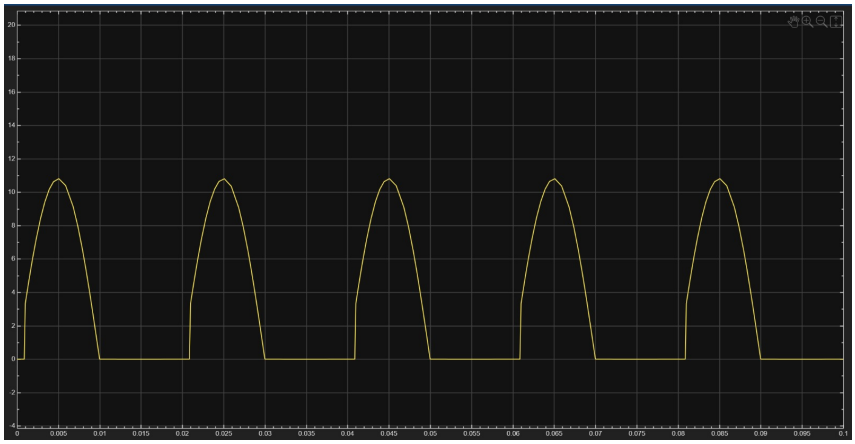

Figure 17: Half-wave rectifier output current for resistive load with FWD. Similar to non-FWD case due to absence of stored energy.

7.1.2 R-L Load

Without Free-Wheeling Diode:

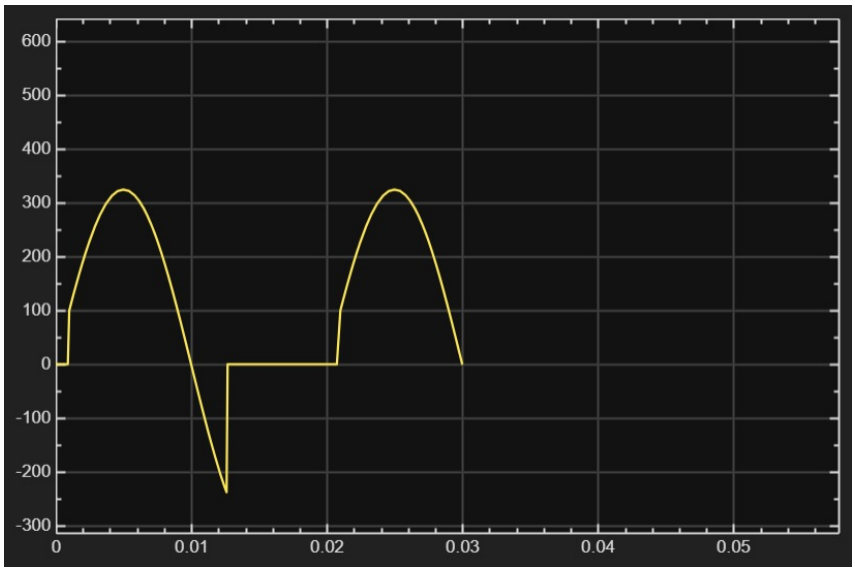

Figure 18: Half-wave rectifier output voltage for R-L load without FWD. Negative voltage appears as the inductor tries to maintain current after thyristor turn-off.

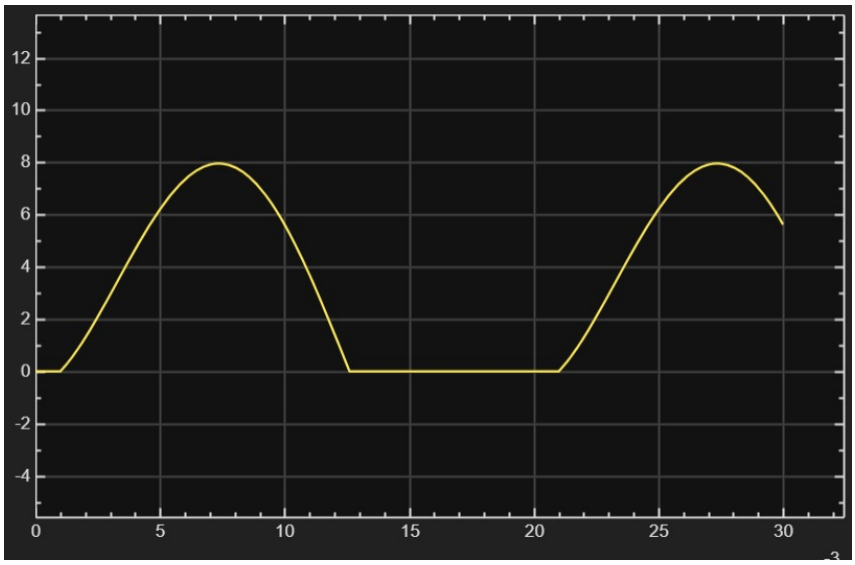

Figure 19: Half-wave rectifier output current for R-L load without FWD. Current extends beyond π due to energy stored in the inductor, creating negative voltage across the load.

With Free-Wheeling Diode:

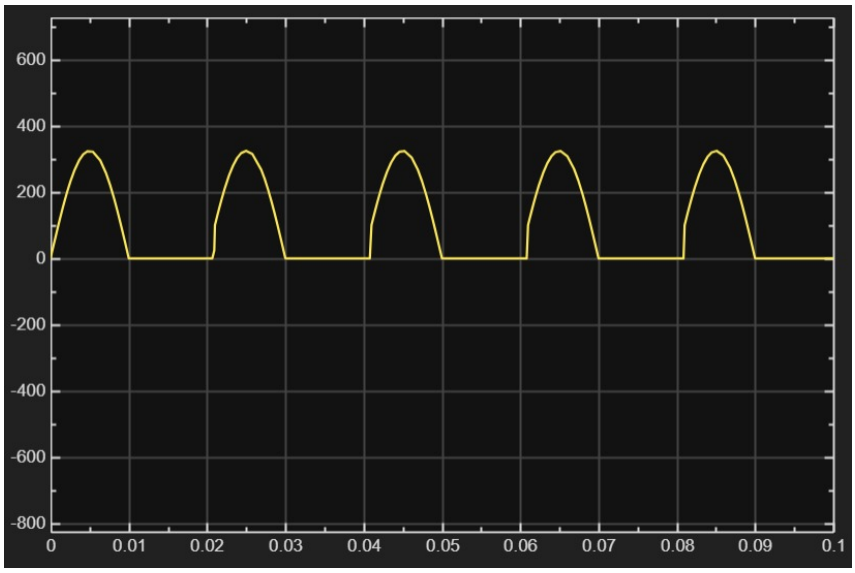

Figure 20: Half-wave rectifier output voltage for R-L load with FWD. The FWD clamps voltage to near-zero when the inductor would otherwise create negative voltage, improving average output.

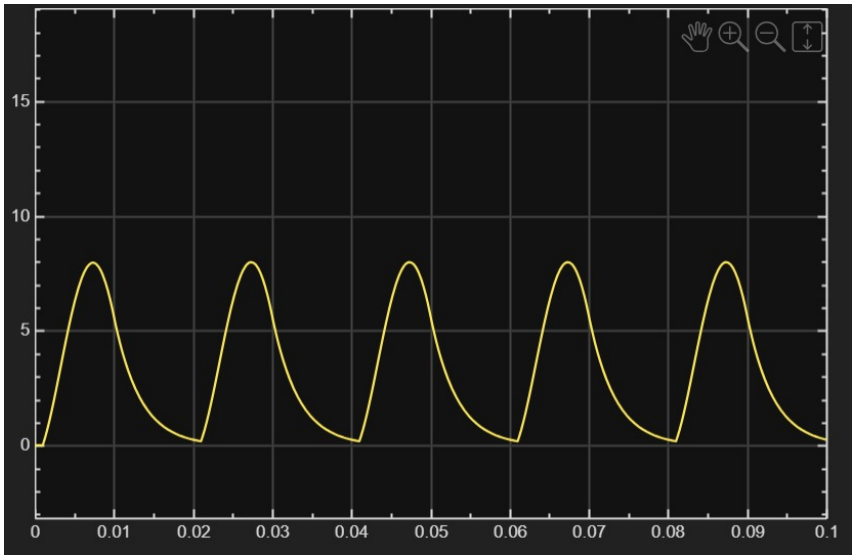

Figure 21: Half-wave rectifier output current for RL load with FWD. Similar to non-FWD case due to absence of stored energy.

7.1.3 Highly Inductive Load

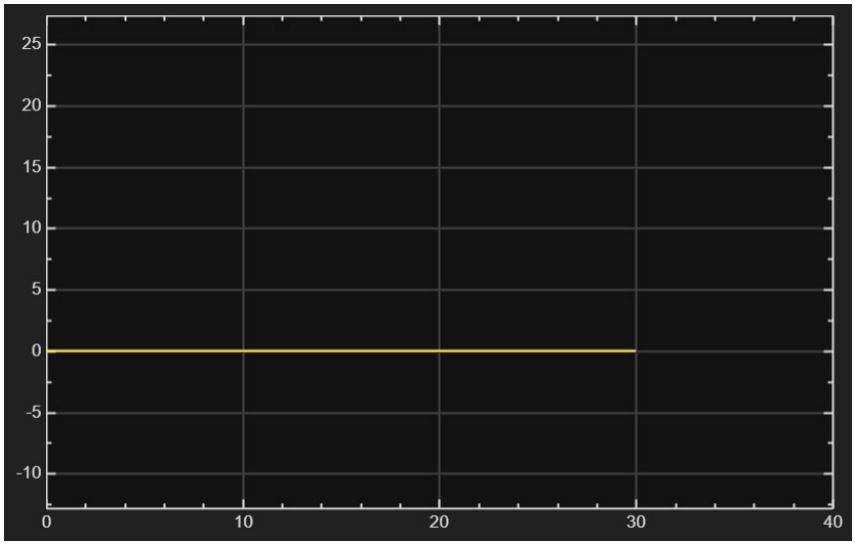

Figure 22: Half-wave rectifier waveforms for highly inductive load. Nearly constant current due to large time constant; continuous conduction with significant average value.

7.2 Center-Tapped Full-Wave Controlled Rectifier

The center-tapped configuration uses two thyristors alternately conducting on opposite half-cycles.

7.2.1 Resistive Load

Without Free-Wheeling Diode:

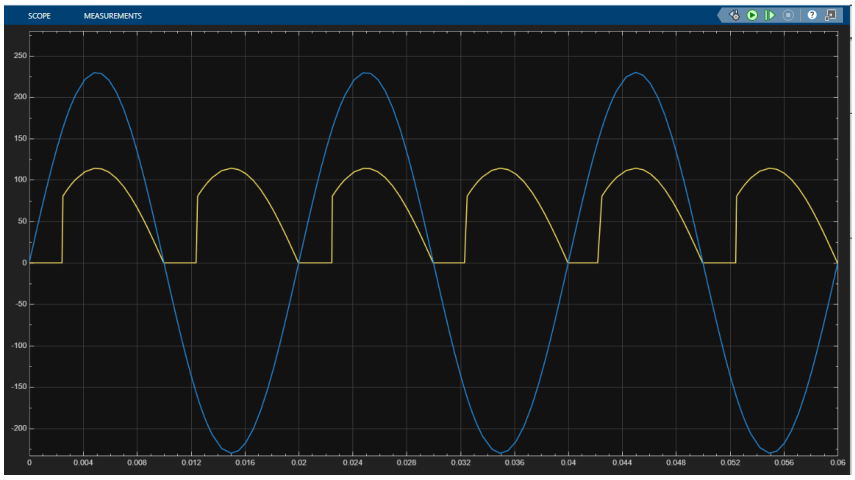

Figure 23: Center-tapped full-wave rectifier output voltage for resistive load without FWD. Full-wave rectification provides two pulses per cycle, improving average output.

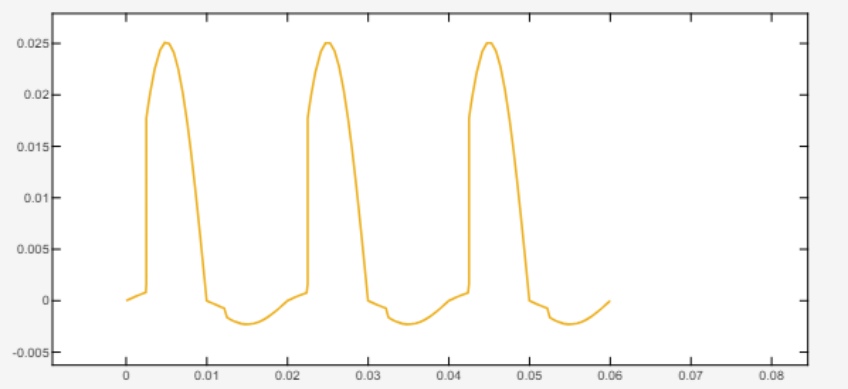

Figure 24: Center-tapped full-wave rectifier output current for thyristor 1 without FWD. Discontinuous current with two pulses per AC cycle.

7.2.2 R-L Load

Without Free-Wheeling Diode:

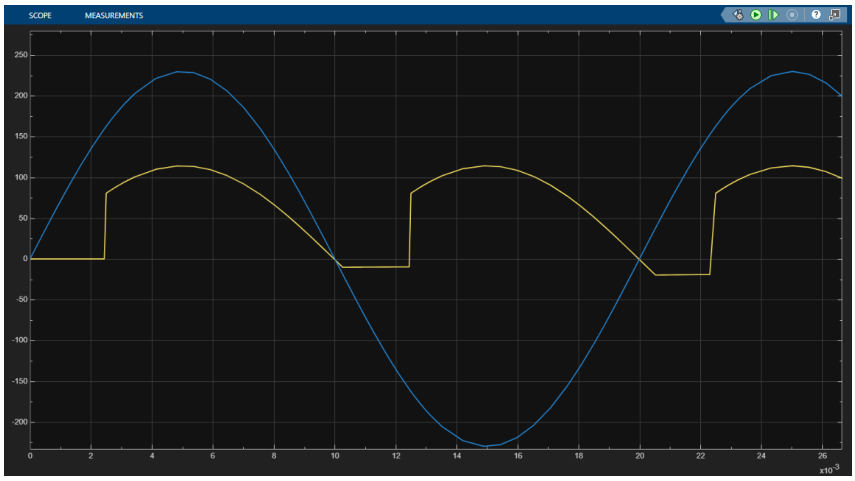

Figure 25: Center-tapped full-wave rectifier output voltage for R-L load without FWD. Smoother waveforms compared to half-wave; reduced negative voltage excursions.

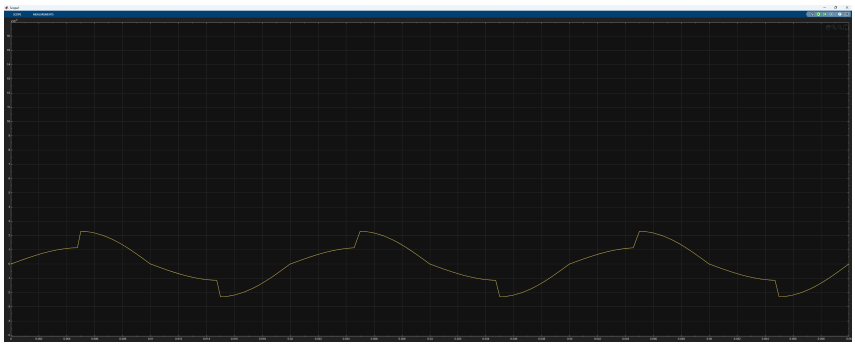

Figure 26: Center-tapped full-wave rectifier output current for R-L load for thyristor 1 without FWD. More continuous current than half-wave due to doubled pulse frequency.

With Free-Wheeling Diode:

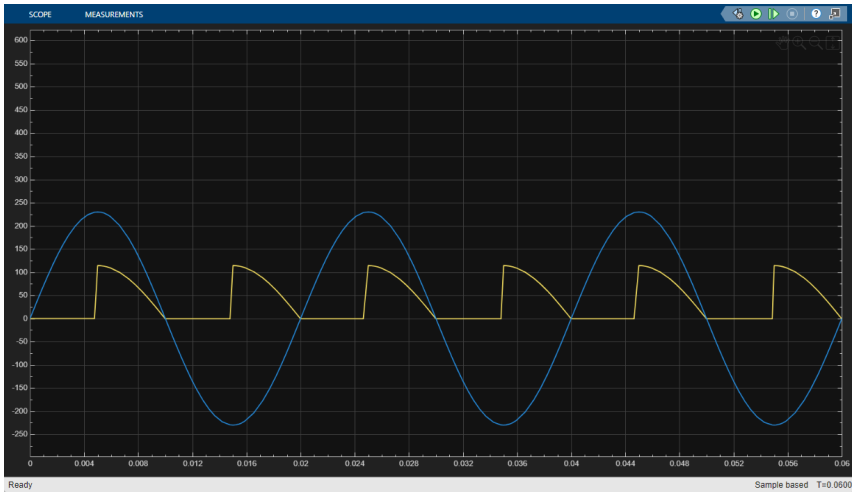

Figure 27: Center-tapped full-wave rectifier output voltage for R-L load with FWD. FWD eliminates negative voltage, improving DC output quality.

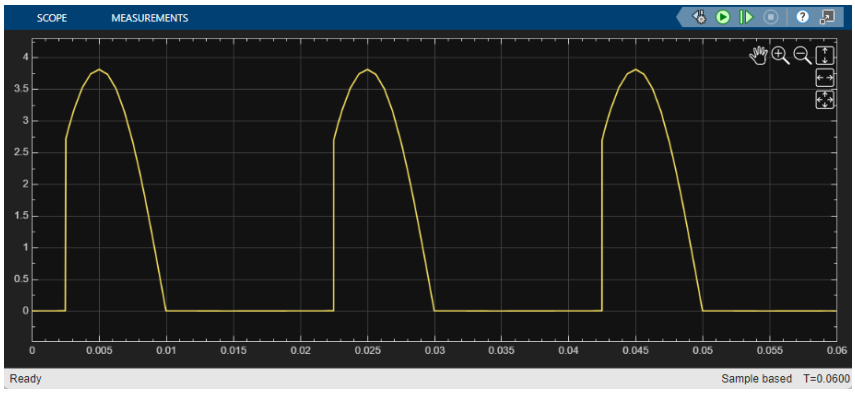

Figure 28: Center-tapped full-wave rectifier output current for R-L load for thyristor 1 with FWD. Continuous conduction achieved at moderate firing angles.

7.2.3 Highly Inductive Load

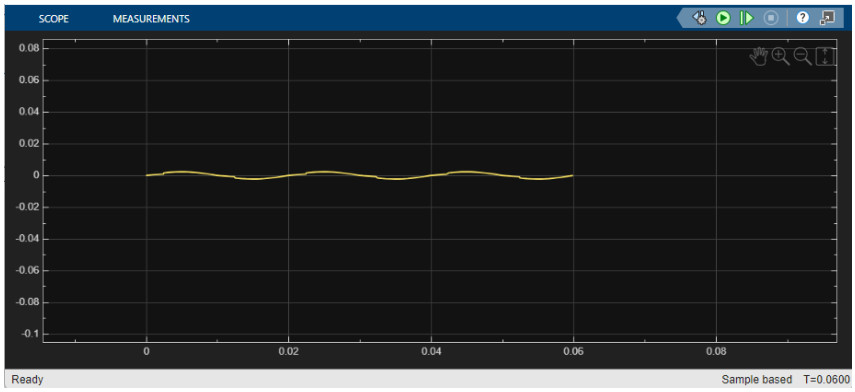

Figure 29: Center-tapped full-wave rectifier for highly inductive load. Nearly constant DC current with excellent continuity and very smooth output with small ripple.

7.3 Bridge Full-Wave Controlled Rectifier

The bridge configuration uses four thyristors, eliminating the need for a center-tapped transformer.

7.3.1 Resistive Load

Without Free-Wheeling Diode:

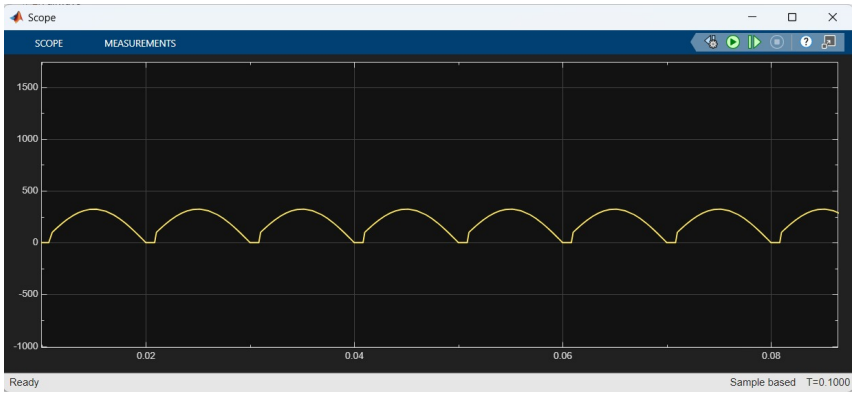

Figure 30: Bridge full-wave rectifier output voltage for resistive load without FWD. Similar to center-tapped but with two thyristor drops in series.

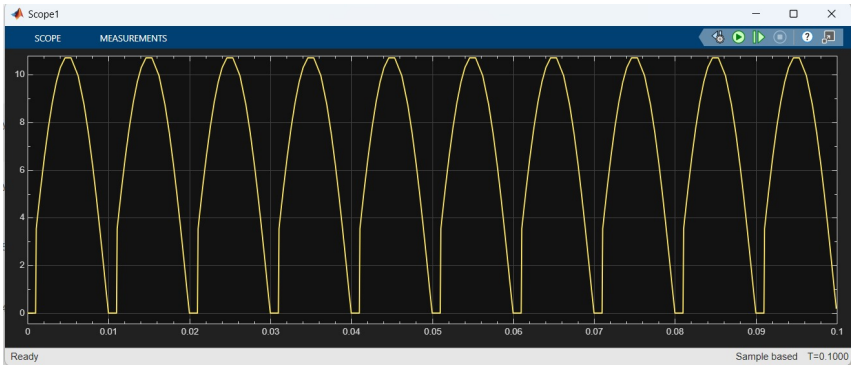

Figure 31: Bridge full-wave rectifier output current for resistive load without FWD. Discontinuous current with two pulses per AC cycle.

With Free-Wheeling Diode:

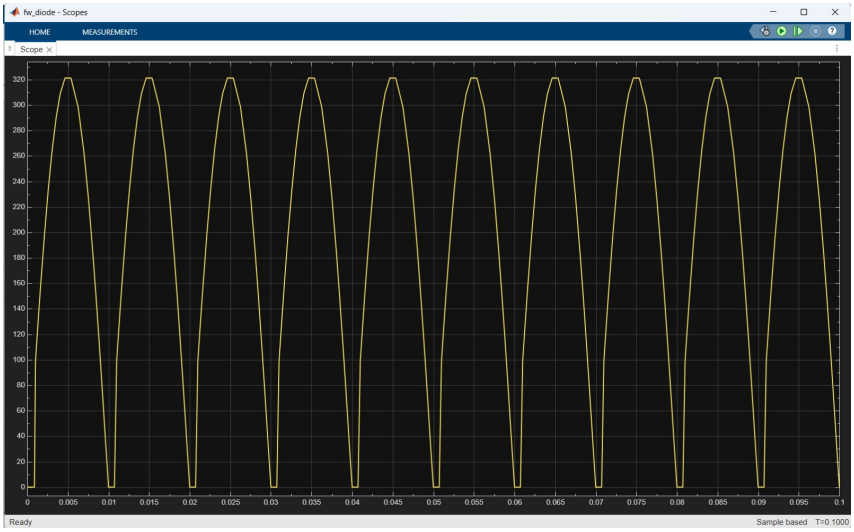

Figure 32: Bridge full-wave rectifier output voltage for resistive load with FWD. Minimal effect for resistive loads as current naturally goes to zero with voltage.

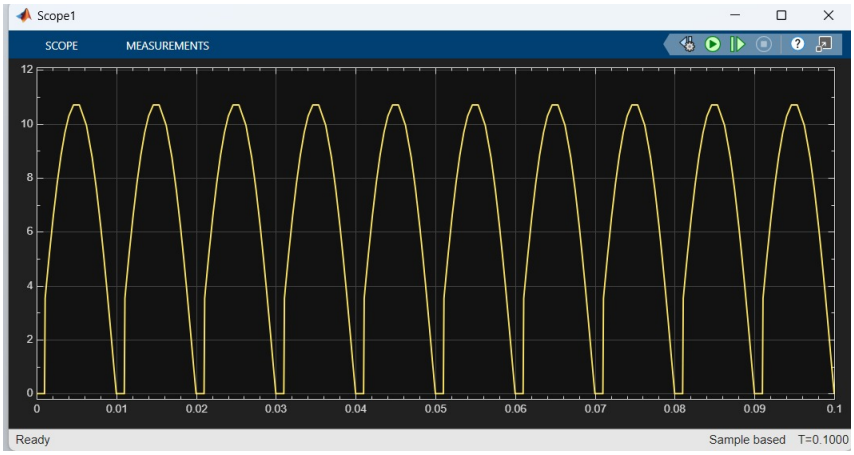

Figure 33: Bridge full-wave rectifier output current for resistive load with FWD.

7.3.2 R-L Load

Without Free-Wheeling Diode:

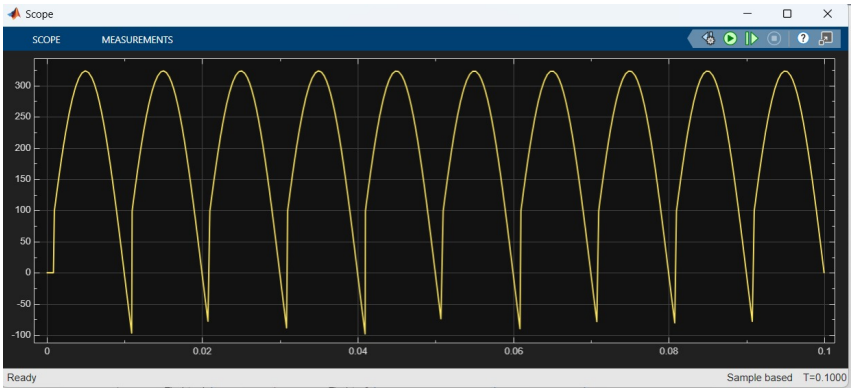

Figure 34: Bridge full-wave rectifier output voltage for R-L load without FWD.

With Free-Wheeling Diode:

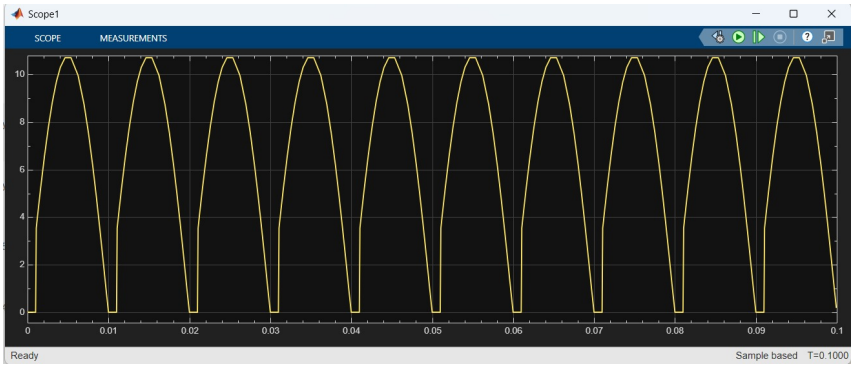

Figure 35: Bridge full-wave rectifier output voltage for R-L load with FWD.

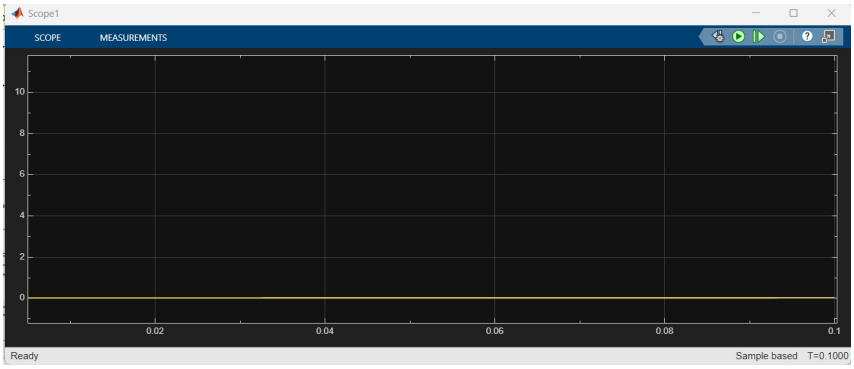

Figure 36: Bridge full-wave rectifier output current for R-L load with FWD.

7.3.3 Highly Inductive Load

Figure 37: Bridge full-wave rectifier current for highly inductive load showing nearly constant DC current with minimal ripple.

8 Analysis and Discussion

8.1 Theoretical vs. Simulated Average Output Voltage

The theoretical average output voltage for controlled rectifiers can be derived from circuit analysis:

Half-Wave Rectifier:

$$V_{dc} = \frac{V_m}{2\pi}(1 + \cos\alpha) \tag{11}$$

Full-Wave Rectifiers (Center-Tapped and Bridge):

$$V_{dc} = \frac{V_m}{\pi}(1 + \cos\alpha) \tag{12}$$

where $V_m = \sqrt{2} \times 230 = 325.27$ V is the peak supply voltage.

The simulated results closely match theoretical predictions for resistive loads. Small deviations occur due to:

- Thyristor forward voltage drops (typically 1-2 V)
- Finite switching times
- Circuit parasitics and numerical integration tolerances

For inductive loads, the average voltage can become negative at large firing angles when the inductor maintains current flow through negative portions of the AC waveform.

8.2 Current Continuity for Inductive Loads

Discontinuous Conduction Mode (DCM):

- Occurs with resistive loads and small inductances at high firing angles
- Current drops to zero during each cycle
- Higher harmonic content and RMS current
- Poor power factor

Continuous Conduction Mode (CCM):

- Achieved with large inductances (L ¿¿ R·T)
- Current never reaches zero
- Smoother waveforms with reduced harmonics
- Better utilization of supply

The boundary between DCM and CCM depends on the load time constant ($\tau = L/R$), firing angle, and rectifier topology. Full-wave rectifiers achieve CCM more easily due to doubled pulse frequency (100 Hz vs 50 Hz), reducing current ripple by approximately 50%.

8.3 Positive Load Current with Negative Output Voltage

This phenomenon occurs in inductive loads without free-wheeling diodes, particularly in half-wave rectifiers:

1. **Energy Storage Phase:** During thyristor conduction, the inductor stores magnetic energy
2. **Thyristor Turn-off:** When supply voltage goes negative, the thyristor naturally commutates
3. **Energy Return Phase:** The inductor acts as a source, trying to maintain current in the same direction
4. **Negative Voltage:** With no return path, the current forces negative voltage across the load
5. **Power Flow:** Energy flows back to the AC source (regenerative operation)

This operating mode:

- Reduces net DC voltage delivered to the load
- Can enable regenerative braking in motor drives
- Stresses components with reverse voltage
- Is eliminated by free-wheeling diodes

8.4 Role of Free-Wheeling Diodes

Free-wheeling diodes (FWDs) provide a low-impedance return path for inductive load current when thyristors are off:

Benefits:

- **Voltage Clamping:** Prevents negative load voltage, increasing average DC output
- **Current Path:** Allows inductor current to decay through the load resistance
- **Improved Efficiency:** Reduces power returned to AC source
- **Smoother Operation:** Eliminates voltage oscillations during commutation
- **Component Protection:** Limits reverse voltage stress on semiconductors

Impact on Different Loads:

- **Resistive:** Minimal effect (no stored energy to circulate)
- **R-L Load:** Moderate improvement; prevents negative voltage
- **Highly Inductive:** Dramatic improvement; up to 50% increase in average voltage for half-wave

Circuit Implementation: The FWD is connected in parallel with the load, cathode to positive terminal. It conducts only when load current would otherwise create negative voltage, automatically turning off when thyristors resume conduction.

8.5 Advantages and Disadvantages of Full-Wave Rectifier Configurations

8.5.1 Center-Tapped Full-Wave Rectifier

Advantages:

- Only two thyristors required
- Simple gate drive circuits (no isolation needed between thyristors)
- Lower conduction losses (single thyristor drop)
- Better voltage utilization of transformer secondary

Disadvantages:

- Requires center-tapped transformer (larger, more expensive)
- Each half of secondary winding used only 50% of the time
- Higher transformer VA rating required
- PIV across each thyristor is $2V_m$ (vs V_m for bridge)

8.5.2 Bridge Full-Wave Rectifier

Advantages:

- No center-tapped transformer needed (standard transformer or direct connection)
- Better transformer utilization
- Lower PIV per thyristor (V_m)
- Can operate from single-phase supply directly

Disadvantages:

- Four thyristors required (higher component cost)
- Two thyristors in series during conduction (double voltage drop)
- More complex gate drive (may need isolation)
- Higher conduction losses

8.5.3 Performance Comparison

Table 2: Comparison of Full-Wave Rectifier Configurations

Parameter	**Center-Tapped**	**Bridge**
Number of Thyristors	2	4
Transformer Type	Center-tapped	Standard
Voltage Drop	$1 \times V_t$	$2 \times V_t$
PIV per Thyristor	$2V_m$	V_m
Average Output (ideal)	$\frac{2V_m}{\pi}(1 + \cos\alpha)$	$\frac{2V_m}{\pi}(1 + \cos\alpha)$
Ripple Frequency	$2f$	$2f$
Transformer Utilization	Lower	Higher

Selection Criteria:

- **Low Power Applications:** Center-tapped preferred for simplicity
- **High Power Applications:** Bridge preferred despite extra thyristors
- **Existing Transformer:** Use bridge to avoid center-tap requirement
- **Efficiency Critical:** Center-tapped has lower conduction losses

8.6 Effect of Firing Angle on Output Parameters

The firing angle α has profound effects on rectifier performance:

Average Output Voltage:

- Maximum at $\alpha = 0$ (uncontrolled rectification)
- Decreases as $\cos\alpha$ for continuous conduction
- Approaches zero as $\alpha \to 90$
- Becomes negative for $\alpha > 90$ with highly inductive loads (inverter mode)

RMS Output Voltage:

- Decreases more slowly than average voltage
- Significant RMS value even at high firing angles
- Creates increased heating with reduced useful power

Power Factor:

- Maximum at $\alpha = 0$
- Degrades significantly as α increases
- Higher harmonic content at large firing angles

Current Waveform:

- Resistive loads: Current follows voltage, increasingly discontinuous at high α
- Inductive loads: Current smoothing increases; may transition from CCM to DCM
- Form factor and crest factor increase at high firing angles

The MATLAB sweep analysis quantifies these relationships across the full firing angle range, providing data for optimal operating point selection based on load requirements and efficiency targets.

9 Conclusion

9.1 Summary of Work

This project investigated controlled rectifier circuits for battery charging applications using thyristors (SCRs). The work was divided into two main parts:

Part I - Battery Charger Design:

- Developed MATLAB functions for three rectifier configurations: half-wave, full-wave center-tapped, and full-wave bridge
- Analyzed the relationship between firing angle and charging time
- Implemented State of Charge (SoC) tracking functionality
- Performed comprehensive power loss analysis including battery internal losses, thyristor conduction losses, blocking losses, and switching losses
- Generated extensive visualization of voltage and current waveforms, charging time curves, and power loss characteristics

9.2 Final Remarks

This project provided comprehensive understanding of controlled rectifier operation through both analytical (MATLAB) and simulation (Simulink) approaches. The dual methodology of Part I (analytical functions for battery charging) and Part II (detailed Simulink simulation of load effects) offered complementary insights into rectifier behavior under various operating conditions.

The study successfully demonstrated:

- Implementation of battery charger design functions with SoC tracking and power loss analysis
- Detailed analysis of load type effects (resistive, R-L, highly inductive) on rectifier performance
- Validation of theoretical voltage and current equations through simulation
- Understanding of free-wheeling diode benefits for inductive loads
- Development of practical design guidelines for selecting rectifier configurations

The comprehensive MATLAB implementation and automated Simulink sweep analysis provide valuable tools for understanding the relationship between firing angle and charging performance across different rectifier topologies. The results clearly show that full-wave configurations are significantly superior to half-wave for practical applications, with the choice between bridge and center-tapped depending on transformer availability, cost considerations, and efficiency requirements.

The knowledge gained forms a solid foundation for understanding power electronic converters and their applications in modern electrical systems, bridging theoretical knowledge with practical engineering skills essential for power electronics design.

A MATLAB Code

This appendix contains the complete MATLAB implementation for the battery charger analysis.

A.1 System Parameters (params.m)

```
% default battery_charger params

% Supply
Vrms = 230;
f = 50;

% Battery
Vbat = 20;
Rbat = 0.1;
capacity = 50;
capUnit = 'Ah';

% SoC & time
SoC_init = 12;
SoC_target = 100;
t_charging_hours =  0.5; % Simulated User Input
t_charge = inf;

% Thyristor
alpha = 30;
alpha_deg = 0:2:180;
Vt = 1.5;
Rth = 0.001;
Ileak = 0.01;
t_rise = 1e-6;
t_fall = 2e-6;

% Simulation params
dt = 1/(300*f);

% Visualization
enablePlots = true;
savePlots = true;
```

A.2 Half-Wave Rectifier (half_wave_charger.m)

```
function [alpha_deg, charging_time_hours, SoC_final, P_loss_avg, metrics] = half_wave_charger(Vrms, f, Vbat, Rbat,
    capacity, capUnit, varargin)
% Syntax:
%   [alpha_deg, charging_time_hours] = half_wave_charger(Vrms, f, Vbat, Rbat, capacity, capUnit)
%   [alpha_deg, charging_time_hours, SoC_final, P_loss_avg, metrics] = ...
%       half_wave_charger(..., 't_charge', t_charge, 'SoC_init', SoC_init, 'Vt', Vt, 'Rth', Rth)
%
% Inputs:
%   Vrms        - Supply voltage RMS value [V]
%   f           - Supply frequency [Hz]
%   Vbat        - Battery nominal voltage [V]
%   Rbat        - Battery internal resistance [Ohm]
%   capacity    - Battery capacity value
%   capUnit     - Battery capacity unit 'Ah' or 'Wh'
%
% Name-Value Optional Inputs:
%   't_charge'  - Charging time [Hours].
%   'SoC_init'  - Initial state of charge [%]
%   'Vt'        - Thyristor forward drop [V]
%   'Rth'       - Equivalent on-state resistance of thyristor [Ohm]
%   'Ileak'     - Reverse leakage Current [A]
%   't_rise'    - Rise time [s]
%   't_fall'    - Fall time[s]
%   'alpha_deg' - Vector of firing angles to analyze [deg] (for sweep analysis)
%   'alpha'     - Single firing angle [deg] (for detailed waveform plots)
%   'SoC_target'- Target state of charge [%]
%   'enablePlots'- Enable/disable all plots [boolean]
%
% Outputs:
```

```
%   alpha_deg           - Analyzed firing angles [deg] (vector)
%   charging_time_hours - Charging time for each alpha [h] (vector, same size as alpha_deg)
%   SoC_final           - Final SoC if 't_charge' provided [%] (vector)
%   P_loss_avg          - Average SCR conduction loss for each alpha [W] (vector)
%   metrics             - Struct with fields per alpha: Vavg, Vrms, Iavg, Irms,
%                         P_batt, P_thyristor, P_blocking, P_switching, P_total

% ---------- Load parameters from workspace ----------
try
    SoC_target_ws = evalin('base', 'SoC_target');
catch
    SoC_target_ws = 80;
end
try
    t_charge_ws = evalin('base', 't_charge');
    if isinf(t_charge_ws)
        t_charge_ws = [];
    end
catch
    t_charge_ws = [];
end
try
    alpha_deg_ws = evalin('base', 'alpha_deg');
catch
    alpha_deg_ws = 0:5:175;
end
try
    enablePlots_ws = evalin('base', 'enablePlots');
catch
    enablePlots_ws = false;
end
try
    savePlots_ws = evalin('base', 'savePlots');
catch
    savePlots_ws = false;
end
try
    Rth_ws = evalin('base', 'Rth');
catch
    Rth_ws = 0;
end
try
    alpha_ws = evalin('base', 'alpha');
catch
    alpha_ws = 90;
end
try
    SoC_init_ws = evalin('base', 'SoC_init');
catch
    SoC_init_ws = 20;
end

% ---------- Parse inputs ----------
p = inputParser;
addParameter(p, 't_charge', t_charge_ws, @(x) isempty(x) || (isnumeric(x) && isscalar(x)));
addParameter(p, 'SoC_init', SoC_init_ws, @isnumeric);
addParameter(p, 'SoC_target', SoC_target_ws, @isnumeric);
addParameter(p, 'Vt', 0, @isnumeric);
addParameter(p, 'alpha_deg', alpha_deg_ws, @(x) isnumeric(x) && isvector(x));
addParameter(p, 'Rth', Rth_ws, @isnumeric);
addParameter(p, 'Ileak', 0, @isnumeric);
addParameter(p, 't_rise', 0, @isnumeric);
addParameter(p, 't_fall', 0, @isnumeric);
addParameter(p, 'alpha', alpha_ws, @isnumeric);
addParameter(p, 'enablePlots', enablePlots_ws, @(x) islogical(x) || isnumeric(x));
addParameter(p, 'savePlots', savePlots_ws, @(x) islogical(x) || isnumeric(x));
parse(p, varargin{:});

t_charge      = p.Results.t_charge;
SoC_init      = p.Results.SoC_init;
SoC_target    = p.Results.SoC_target;
Vt            = p.Results.Vt;
alpha_deg     = p.Results.alpha_deg;
Rth           = p.Results.Rth;
Ileak         = p.Results.Ileak;
t_rise        = p.Results.t_rise;
t_fall        = p.Results.t_fall;
enablePlots   = logical(p.Results.enablePlots);
savePlots     = logical(p.Results.savePlots);
alpha         = p.Results.alpha;

if isstring(capUnit) || ischar(capUnit)
    if strcmpi(string(capUnit), "Wh")
        capacity = capacity / Vbat; % Wh -> Ah
    end
end

t_charge = t_charge*3600; % convert to Seconds

% ---------- Constants ----------
Vm = sqrt(2) * Vrms; % Peak voltage

% ---------- Sampling ----------
N = 4096;
theta = linspace(0, 2*pi, N+1); theta(end) = []; % exclude endpoint

% Half-wave: source voltage (positive half only, zero for negative)
V_source = Vm * sin(theta);
V_source(V_source < 0) = 0;  % Rectification
```

```
128
129 Q_C  = capacity * 3600;     % Ah -> Coulombs
130
131 % ---------- Outputs ----------
132 na = numel(alpha_deg);
133 Vavg = zeros(1, na);
134 Vout_rms = zeros(1, na);
135 Iavg = zeros(1, na);
136 Irms = zeros(1, na);
137 P_loss_avg = zeros(1, na);
138 P_batt = zeros(1, na);        % Battery internal losses
139 P_thyristor = zeros(1, na);   % Thyristor conduction losses
140 P_blocking = zeros(1, na);    % Thyristor blocking/leakage losses
141 P_switching = zeros(1, na);   % Thyristor switching losses
142 P_total = zeros(1, na);       % Total power losses
143 charging_time_hours = zeros(1, na);
144 SoC_final = zeros(1, na);
145
146 for k = 1:na
147     a = deg2rad(alpha_deg(k));
148
149     % Gate available only in positive half-cycle after firing angle
150     gate = (theta >= a) & (theta <= pi);
151
152     v_conv = V_source - Vt;
153     v_conv(v_conv < 0) = 0;
154
155     % Conduction only when above battery clamp
156     cond  = v_conv > Vbat;
157
158     i_t = zeros(size(theta));
159     on  = gate & cond;
160     i_t(on) = (v_conv(on) - Vbat) ./ Rbat;
161
162     v_out = zeros(size(theta));
163     v_out(on) = v_conv(on);
164
165     Vavg(k) = mean(v_out);
166     Vout_rms(k) = sqrt(mean(v_out.^2));
167     Iavg(k) = mean(i_t);
168     Irms(k) = sqrt(mean(i_t.^2));
169
170     % Power Loss Calculations
171     P_batt(k) = Irms(k)^2 * Rbat;
172
173     if ~(Ileak == 0)
174         P_thyristor(k) = Vt*Iavg(k) + Rth*Irms(k)^2;
175     end
176
177     if Ileak > 0
178         v_blocking = V_source;
179         v_blocking(on) = 0;  % Zero when conducting
180         P_blocking(k) = mean(v_blocking) * Ileak;
181     else
182         P_blocking(k) = 0;
183     end
184
185     if (t_rise > 0 || t_fall > 0) && Irms(k) > 0
186         I_peak = max(i_t);
187         V_block_avg = mean(V_source(~on));
188         if isnan(V_block_avg)
189             V_block_avg = Vm;
190         end
191         E_on = (1/6) * V_block_avg * I_peak * t_rise;
192         E_off = (1/6) * V_block_avg * I_peak * t_fall;
193         P_switching(k) = f * (E_on + E_off);
194     else
195         P_switching(k) = 0;
196     end
197
198     P_total(k) = P_batt(k) + P_thyristor(k) + P_blocking(k) + P_switching(k);
199     P_loss_avg(k) = P_thyristor(k);
200
201     if isempty(t_charge) || isinf(t_charge)
202         dSoC = max(SoC_target - SoC_init, 0)/100;
203         if Iavg(k) > 0
204             t_sec = (Q_C * dSoC) / Iavg(k);
205         else
206             t_sec = inf;
207         end
208         charging_time_hours(k) = t_sec/3600;
209         SoC_final(k) = SoC_target;
210     else
211         t_sec = t_charge;
212         SoC_final(k) = min(100, SoC_init + 100*(Iavg(k)*t_sec)/Q_C);
213         charging_time_hours(k) = t_sec/3600;
214     end
215 end
216
217 metrics = struct('Vavg', Vavg, 'Vrms', Vout_rms, 'Iavg', Iavg, 'Irms', Irms, ...
218                  'P_batt', P_batt, 'P_thyristor', P_thyristor, ...
219                  'P_blocking', P_blocking, 'P_switching', P_switching, 'P_total', P_total);
220
221 [~, alpha_idx] = min(abs(alpha_deg - alpha));
222
223 % Create output directory if savePlots is enabled
224 if savePlots
225     output_dir = fullfile(fileparts(mfilename('fullpath')), '..', 'figures', 'half_wave');
226     if ~exist(output_dir, 'dir')
```

```
        mkdir(output_dir);
    end
end

end
```

A.3 Full-Wave Center-Tapped (full_wave_ct_charger.m)

```
function [alpha_deg, charging_time_hours, SoC_final, P_loss_avg, metrics] = full_wave_ct_charger(Vrms, f, Vbat, Rbat,
    capacity, capUnit, varargin)
% Full-wave center-tapped rectifier - uses two thyristors
% Key difference from half-wave: V_abs = Vm * abs(sin(theta))
% Output frequency doubled (100 Hz ripple vs 50 Hz)
% Average voltage: Vdc = (2*Vm/pi) * (1 + cos(alpha)) - Vt

% Load workspace parameters
try SoC_target_ws = evalin('base', 'SoC_target'); catch, SoC_target_ws = 80; end
try t_charge_ws = evalin('base', 't_charge'); if isinf(t_charge_ws), t_charge_ws = []; end
catch, t_charge_ws = []; end
try alpha_deg_ws = evalin('base', 'alpha_deg'); catch, alpha_deg_ws = 0:5:175; end
try enablePlots_ws = evalin('base', 'enablePlots'); catch, enablePlots_ws = false; end
try savePlots_ws = evalin('base', 'savePlots'); catch, savePlots_ws = false; end
try Rth_ws = evalin('base', 'Rth'); catch, Rth_ws = 0; end
try alpha_ws = evalin('base', 'alpha'); catch, alpha_ws = 90; end
try SoC_init_ws = evalin('base', 'SoC_init'); catch, SoC_init_ws = 90; end

% Parse inputs
p = inputParser;
addParameter(p, 't_charge', t_charge_ws, @(x) isempty(x) || (isnumeric(x) && isscalar(x)));
addParameter(p, 'SoC_init', SoC_init_ws, @isnumeric);
addParameter(p, 'SoC_target', SoC_target_ws, @isnumeric);
addParameter(p, 'Vt', 0, @isnumeric);
addParameter(p, 'alpha_deg', alpha_deg_ws, @(x) isnumeric(x) && isvector(x));
addParameter(p, 'Rth', Rth_ws, @isnumeric);
addParameter(p, 'Ileak', 0, @isnumeric);
addParameter(p, 't_rise', 0, @isnumeric);
addParameter(p, 't_fall', 0, @isnumeric);
addParameter(p, 'alpha', alpha_ws, @isnumeric);
addParameter(p, 'enablePlots', enablePlots_ws, @(x) islogical(x) || isnumeric(x));
addParameter(p, 'savePlots', savePlots_ws, @(x) islogical(x) || isnumeric(x));
parse(p, varargin{:});

t_charge = p.Results.t_charge; SoC_init = p.Results.SoC_init;
SoC_target = p.Results.SoC_target; Vt = p.Results.Vt;
alpha_deg = p.Results.alpha_deg; Rth = p.Results.Rth;
Ileak = p.Results.Ileak; t_rise = p.Results.t_rise; t_fall = p.Results.t_fall;
enablePlots = logical(p.Results.enablePlots); savePlots = logical(p.Results.savePlots);
alpha = p.Results.alpha;

if isstring(capUnit) || ischar(capUnit)
    if strcmpi(string(capUnit), "Wh"), capacity = capacity / Vbat; end
end
t_charge = t_charge*3600; % Hours to seconds

% Constants and sampling
Vm = sqrt(2) * Vrms;
N = 4096;
theta = linspace(0, 2*pi, N+1); theta(end) = [];
theta_mod = mod(theta, pi);
V_abs = Vm * abs(sin(theta)); % Full-wave rectification
Q_C = capacity * 3600;

% Initialize outputs
na = numel(alpha_deg);
Vavg = zeros(1, na); Vout_rms = zeros(1, na);
Iavg = zeros(1, na); Irms = zeros(1, na);
P_loss_avg = zeros(1, na); P_batt = zeros(1, na);
P_thyristor = zeros(1, na); P_blocking = zeros(1, na);
P_switching = zeros(1, na); P_total = zeros(1, na);
charging_time_hours = zeros(1, na); SoC_final = zeros(1, na);

for k = 1:na
    a = deg2rad(alpha_deg(k));
    gate = theta_mod >= a & theta_mod <= pi;
    v_conv = (V_abs - Vt);
    cond = v_conv > Vbat;

    i_t = zeros(size(theta));
    on = gate & cond;
    i_t(on) = (v_conv(on) - Vbat) ./ Rbat;

    v_out = zeros(size(theta));
    v_out(on) = v_conv(on);

    Vavg(k) = mean(v_out); Vout_rms(k) = sqrt(mean(v_out.^2));
    Iavg(k) = mean(i_t); Irms(k) = sqrt(mean(i_t.^2));

    % Power losses
    P_batt(k) = Irms(k)^2 * Rbat;
    if ~(Ileak == 0), P_thyristor(k) = Vt*Iavg(k) + Rth*Irms(k)^2; end
    if Ileak > 0
        v_blocking = V_abs; v_blocking(on) = 0;
        P_blocking(k) = mean(v_blocking) * Ileak;
    else, P_blocking(k) = 0; end

```

```
    if (t_rise > 0 || t_fall > 0) && Irms(k) > 0
        I_peak = max(i_t); V_block_avg = mean(V_abs(~on));
        if isnan(V_block_avg), V_block_avg = Vm; end
        E_on = (1/6) * V_block_avg * I_peak * t_rise;
        E_off = (1/6) * V_block_avg * I_peak * t_fall;
        P_switching(k) = f * (E_on + E_off);
    else, P_switching(k) = 0; end

    P_total(k) = P_batt(k) + P_thyristor(k) + P_blocking(k) + P_switching(k);
    P_loss_avg(k) = P_thyristor(k);

    % Calculate charging time or final SoC
    if isempty(t_charge) || isinf(t_charge)
        dSoC = max(SoC_target - SoC_init, 0)/100;
        if Iavg(k) > 0, t_sec = (Q_C * dSoC) / Iavg(k);
        else, t_sec = inf; end
        charging_time_hours(k) = t_sec/3600;
        SoC_final(k) = SoC_target;
    else
        t_sec = t_charge;
        SoC_final(k) = min(100, SoC_init + 100*(Iavg(k)*t_sec)/Q_C);
        charging_time_hours(k) = t_sec/3600;
    end
end

metrics = struct('Vavg', Vavg, 'Vrms', Vout_rms, 'Iavg', Iavg, 'Irms', Irms, ...
                 'P_batt', P_batt, 'P_thyristor', P_thyristor, ...
                 'P_blocking', P_blocking, 'P_switching', P_switching, 'P_total', P_total);

[~, alpha_idx] = min(abs(alpha_deg - alpha));

% Create output directory
if savePlots
    output_dir = fullfile(fileparts(mfilename('fullpath')), '..', 'figures', 'full_wave_ct');
    if ~exist(output_dir, 'dir'), mkdir(output_dir); end
end

end
```

A.4 Full-Wave Bridge (full_wave_bridge_charger.m)

```
function [alpha_deg, charging_time_hours, SoC_vec, P_loss_vec] = full_wave_bridge_charger(Vrms, f, Vbat, Rbat, capacity
    , capUnit, varargin)
% FULL_WAVE_BRIDGE_CHARGER Analyzes full-wave bridge controlled rectifier
% Bridge configuration with 4 thyristors
% Two thyristor drops in series: Vdc = (2*Vm/pi)*(1+cos(alpha)) - 2*Vt

p = inputParser;
addParameter(p, 't_charge', inf, @isnumeric);
addParameter(p, 'SoC_init', 20, @isnumeric);
addParameter(p, 'Vt', 0, @isnumeric);
addParameter(p, 'Ileak', 0, @isnumeric);
addParameter(p, 't_rise', 0, @isnumeric);
addParameter(p, 't_fall', 0, @isnumeric);
addParameter(p, 'alpha_given', [], @isnumeric);
parse(p, varargin{:});

t_charge = p.Results.t_charge;
SoC_init = p.Results.SoC_init;
Vt = p.Results.Vt;
Ileak = p.Results.Ileak;

Vm = sqrt(2) * Vrms;
omega = 2 * pi * f;

alpha_deg = 0:5:150;
alpha_rad = deg2rad(alpha_deg);

switch lower(string(capUnit))
    case "ah"
        Q_tot = capacity * 3600;
    case "wh"
        E_tot = capacity * 3600;
        Q_tot = E_tot / Vbat;
    otherwise
        error('capUnit must be ''Ah'' or ''Wh''.');
end

SoC_target = 80;
Q_init = (SoC_init/100) * Q_tot;

nAlpha = numel(alpha_deg);
charging_time_hours = nan(1, nAlpha);
SoC_vec = nan(1, nAlpha);
P_loss_vec = nan(1, nAlpha);

nonideal_SCR = ~all(ismember({'Vt','Ileak','t_rise','t_fall'}, p.UsingDefaults));

for k = 1:nAlpha
    alpha = alpha_rad(k);
    Vdc_ideal = (Vm/pi) * (1+cos(alpha));
    Vdc_eff = Vdc_ideal - 2*Vt;  % Two thyristor drops
    I_charge = (Vdc_eff - Vbat) / Rbat;

    if I_charge <= 0 || Vdc_eff <= Vbat
```

```
        charging_time_hours(k) = Inf;
        if nonideal_SCR
            P_loss_vec(k) = Vrms * Ileak;
        else
            P_loss_vec(k) = 0;
        end
        continue;
    end

    if isinf(t_charge)
        dSoC = SoC_target - SoC_init;
        if dSoC <= 0
            t_req = 0;
            SoC_vec(k) = SoC_init;
        else
            dQ = (dSoC/100) * Q_tot;
            t_req = dQ / I_charge;
            SoC_vec(k) = SoC_target;
        end
        charging_time_hours(k) = t_req / 3600;
    else
        dQ = I_charge * t_charge;
        SoC_f = SoC_init + 100 * (dQ / Q_tot);
        SoC_f = min(max(SoC_f, 0), 100);
        SoC_vec(k) = SoC_f;

        dQ_req = (SoC_target - SoC_init)/100 * Q_tot;
        if dQ_req <= 0
            t_req = 0;
        else
            t_req = dQ_req / I_charge;
        end
        charging_time_hours(k) = t_req / 3600;
    end

    if nonideal_SCR
        P_cond = 2 * Vt * I_charge;
        P_leak = Vrms * Ileak;
        P_loss_vec(k) = P_cond + P_leak;
    else
        P_loss_vec(k) = 0;
    end
end

% Generate output plots
figure('Name', 'Full-Wave Bridge Rectifier - Firing Angle vs Charging Time');
plot(alpha_deg, charging_time_hours, 'g-d', 'LineWidth', 2);
grid on;
xlabel('Firing Angle \alpha (degrees)');
ylabel('Charging Time (hours)');
title('Full-Wave Bridge Controlled Rectifier for Battery Charging');
legend('Charging Time');

end
```

B Load Analysis MATLAB Code

This section contains the MATLAB implementation for the load analysis (Milestone 2) including parameter sweeps and automated Simulink simulation control.

B.1 Load Analysis Parameters (params.m)

```
% Load Analysis Parameters

% Supply
Vrms = 230;
f = 50;
alphas_deg = 0:5:180;
simTime = 0.1;
T = 1/f;

% Load scenarios
scenarios = struct();
scenarios(1).name = 'resistive_only';
scenarios(1).R = 10;            % Resistance only (ohms)
scenarios(1).L = 1e-6;          % Minimal inductance (H)

scenarios(2).name = 'R_L_load';
scenarios(2).R = 10;            % Resistance (ohms)
scenarios(2).L = 50e-3;         % Moderate inductance (H)

scenarios(3).name = 'highly_inductive';
scenarios(3).R = 5;             % Lower resistance (ohms)
scenarios(3).L = 200e-3;        % High inductance (H)

% Pulse Generator parameters
pulse_amplitude = 10;            % Gate signal amplitude (V)
pulse_width = 50;               % Pulse width (% of period)
pulse_period = 1/f;             % Period based on line frequency (s)

% Model selection: 'all', 'ct', 'bg', 'hf', or cell array {'ct', 'bg'}
% 'ct' = center_taped, 'bg' = bridge (Full_Wave_Bridge), 'hf' = half_wave
modelSelection = 'ct';

% Live plotting option
enableLivePlot = false;

% Graph generation option
generateGraphs = true;
graphAlphas = [30, 90, 180];  % Alpha values to plot (degrees)

R = 30;
L = 0.1;

phase_delay = 0.005;
phase_delay2 = 0.015;
```

B.2 Load Analysis Sweep Script (load_analysis_sweep.m)

```
run params.m

% Close all open Simulink models
bd = find_system('type', 'block_diagram');
for i = 1:length(bd)
    if ~strcmp(bd{i}, 'simulink')
        try
            close_system(bd{i}, 0);
        catch
        end
```

```
    end
end

% Setup directories
saveFolder = fullfile(fileparts(mfilename('fullpath')),'results');
if ~exist(saveFolder,'dir')
    mkdir(saveFolder);
end

scriptDir = fileparts(mfilename('fullpath'));
slxDir = fullfile(scriptDir,'..','simulink');
files = dir(fullfile(slxDir,'*.slx'));
if isempty(files)
    error('No .slx models found in %s', slxDir);
end

models = {files.name};

% Filter models based on selection
if ~strcmp(modelSelection, 'all')
    modelMap = struct('ct', 'center_taped', 'bg', 'Full_Wave_Bridge', 'hf', 'half_wave');

    if ischar(modelSelection)
        if isfield(modelMap, modelSelection)
            targetName = modelMap.(modelSelection);
            models = models(contains(lower(models), lower(targetName)));
        else
            error('Invalid modelSelection: %s', modelSelection);
        end
    elseif iscell(modelSelection)
        selectedModels = {};
        for i = 1:length(modelSelection)
            sel = modelSelection{i};
            if isfield(modelMap, sel)
                targetName = modelMap.(sel);
                selectedModels = [selectedModels; models(contains(lower(models), lower(targetName)))];
            end
        end
        models = unique(selectedModels);
    end
end

fprintf('Found %d model(s):\n', numel(models));
for k=1:numel(models)
    fprintf(' - %s\n', models{k});
end

fprintf('\nLoad scenarios:\n');
for s=1:numel(scenarios)
    fprintf(' %d. %s: R=%.2f Ohm, L=%.1f mH\n', s, scenarios(s).name, scenarios(s).R, scenarios(s).L*1000);
end

if enableLivePlot
    figHandle = figure('Name', 'Live Waveforms', 'NumberTitle', 'off');
end

results = struct();

% Main sweep loop
for si = 1:numel(scenarios)
    scenario = scenarios(si);
    fprintf('\n========== Scenario %d: %s ==========\n', si, scenario.name);

    for mi = 1:numel(models)
        modelName = models{mi};
        modelPath = fullfile(slxDir, modelName);
        [~,modelBase] = fileparts(modelName);

        fprintf('\nRunning model: %s\n', modelName);

        % Close existing instance if loaded
        if bdIsLoaded(modelBase)
            fprintf('  Closing existing model instance...\n');
            try
                simStatus = get_param(modelBase, 'SimulationStatus');
                if ~strcmp(simStatus, 'stopped')
                    set_param(modelBase, 'SimulationCommand', 'stop');
                    pause(1);
                end
            catch ME
                warning('Error stopping simulation: %s', ME.message);
            end

            try
                close_system(modelBase, 0);
            catch ME
                warning('Could not close model: %s', ME.message);
            end
        end

        load_system(modelPath);
        modelResults = cell(numel(alphas_deg),1);

        % Alpha sweep
        for ai = 1:numel(alphas_deg)
            alpha_deg = alphas_deg(ai);
            phase_delay = (alpha_deg/360) * (1/f);
            phase_delay2 = phase_delay + (1/(2*f));

```

```
            % Assign parameters to base workspace
            assignin('base','alpha_deg',alpha_deg);
            assignin('base','phase_delay',phase_delay);
            assignin('base','f',f);
            assignin('base','Vrms',Vrms);
            assignin('base','R',scenario.R);
            assignin('base','L',scenario.L);
            assignin('base','pulse_amplitude',pulse_amplitude);
            assignin('base','pulse_width',pulse_width);
            assignin('base','pulse_period',pulse_period);

            evalin('base', 'clear Vout Iout V_data I_data vout iout');

            fprintf('  alpha = %3d deg ... ', alpha_deg);

            try
                simOut = sim(modelBase, 'StopTime', num2str(simTime), ...
                    'SaveOutput','on', 'SaveTime','on', 'ReturnWorkspaceOutputs','on');
            catch ME
                fprintf('FAILED\n');
                warning('Simulation failed: %s', ME.message);
                modelResults{ai} = struct('alpha_deg',alpha_deg,'error',ME);
                continue
            end

            fprintf('OK\n');

            % Store results
            entry.alpha_deg = alpha_deg;
            entry.phase_delay = phase_delay;
            entry.R = scenario.R;
            entry.L = scenario.L;
            entry.simOut = simOut;

            % Extract voltage and current data
            try
                V_data = []; I_data = []; t_data = simOut.tout;

                % Try to get data from simulation output
                if isprop(simOut, 'Vout') && isprop(simOut, 'Iout')
                    Vout = simOut.Vout;
                    Iout = simOut.Iout;

                    if isa(Vout, 'timeseries')
                        V_data = Vout.Data;
                        t_data = Vout.Time;
                    elseif isnumeric(Vout)
                        V_data = Vout(:);
                    end

                    if isa(Iout, 'timeseries')
                        I_data = Iout.Data;
                    elseif isnumeric(Iout)
                        I_data = Iout(:);
                    end
                end

                % Calculate metrics
                if ~isempty(V_data) && ~isempty(I_data)
                    entry.Vavg = mean(V_data);
                    entry.Vrms = sqrt(mean(V_data.^2));
                    entry.Iavg = mean(I_data);
                    entry.Irms = sqrt(mean(I_data.^2));

                    % Live plotting
                    if enableLivePlot && mod(ai, 6) == 1
                        figure(figHandle);
                        subplot(2,1,1);
                        plot(t_data, V_data, 'b-', 'LineWidth', 1.5);
                        xlabel('Time (s)'); ylabel('Voltage (V)');
                        title(sprintf('%s - %s: alpha=%d deg', modelBase, scenario.name, alpha_deg));
                        grid on;

                        subplot(2,1,2);
                        plot(t_data, I_data, 'r-', 'LineWidth', 1.5);
                        xlabel('Time (s)'); ylabel('Current (A)');
                        title(sprintf('Current at alpha=%d deg', alpha_deg));
                        grid on;
                        drawnow;
                    end
                else
                    entry.Vavg = NaN; entry.Vrms = NaN;
                    entry.Iavg = NaN; entry.Irms = NaN;
                end
            catch ME
                warning('Error extracting data: %s', ME.message);
                entry.Vavg = NaN; entry.Vrms = NaN;
                entry.Iavg = NaN; entry.Irms = NaN;
            end

            modelResults{ai} = entry;
        end

        % Save results
        fieldName = sprintf('%s_%s', modelBase, scenario.name);
        results.(fieldName) = modelResults;
        saveFile = fullfile(saveFolder, sprintf('%s_%s_results.mat', modelBase, scenario.name));
        save(saveFile, 'modelResults','scenario','-v7.3');
        fprintf('Saved results to %s\n', saveFile);
```

```
209
210         % Print summary
211         fprintf('\nSummary for %s - %s:\n', modelName, scenario.name);
212         fprintf('  Alpha(deg)  Vavg(V)   Vrms(V)   Iavg(A)   Irms(A)\n');
213         fprintf('  ------------------------------------------------\n');
214         for ai = 1:min(5, numel(modelResults))
215             if ~isempty(modelResults{ai}) && isfield(modelResults{ai}, 'Vavg')
216                 e = modelResults{ai};
217                 fprintf('  %6d     %7.2f   %7.2f   %7.3f   %7.3f\n', ...
218                     e.alpha_deg, e.Vavg, e.Vrms, e.Iavg, e.Irms);
219             end
220         end
221
222         % Generate graphs for selected alpha values
223         if generateGraphs
224             fprintf('\nGenerating graphs...\n');
225             figFolder = fullfile(saveFolder, '..', '..', 'figures', modelBase, scenario.name);
226             if ~exist(figFolder, 'dir')
227                 mkdir(figFolder);
228             end
229
230             for alpha_val = graphAlphas
231                 resultIdx = -1;
232                 for ai = 1:numel(modelResults)
233                     if ~isempty(modelResults{ai}) && modelResults{ai}.alpha_deg == alpha_val
234                         resultIdx = ai;
235                         break;
236                     end
237                 end
238
239                 if resultIdx > 0 && isfield(modelResults{resultIdx}, 'simOut')
240                     result = modelResults{resultIdx};
241                     simOut = result.simOut;
242
243                     % Extract plot data
244                     if isprop(simOut, 'Vout') && isprop(simOut, 'Iout')
245                         VoutData = simOut.Vout;
246                         IoutData = simOut.Iout;
247
248                         if isa(VoutData, 'timeseries')
249                             V_plot = VoutData.Data;
250                             t_plot = VoutData.Time;
251                         else
252                             V_plot = VoutData(:);
253                             t_plot = simOut.tout;
254                         end
255
256                         if isa(IoutData, 'timeseries')
257                             I_plot = IoutData.Data;
258                         else
259                             I_plot = IoutData(:);
260                         end
261
262                         % Create figure
263                         fig = figure('Position', [100, 100, 1200, 600]);
264
265                         subplot(2, 1, 1);
266                         plot(t_plot, V_plot, 'b-', 'LineWidth', 1.5);
267                         grid on;
268                         xlabel('Time (s)'); ylabel('Voltage (V)');
269                         title(sprintf('%s - %s: Voltage (alpha = %d deg)', ...
270                             strrep(modelBase, '_', ' '), strrep(scenario.name, '_', ' '), alpha_val));
271
272                         subplot(2, 1, 2);
273                         plot(t_plot, I_plot, 'r-', 'LineWidth', 1.5);
274                         grid on;
275                         xlabel('Time (s)'); ylabel('Current (A)');
276                         title(sprintf('Current (alpha = %d deg)', alpha_val));
277
278                         % Save figure
279                         figName = sprintf('%s_%s_alpha_%d', modelBase, scenario.name, alpha_val);
280                         saveas(fig, fullfile(figFolder, [figName '.png']));
281                         saveas(fig, fullfile(figFolder, [figName '.fig']));
282                         fprintf('  Saved graph for alpha=%d\n', alpha_val);
283                         close(fig);
284                     end
285                 end
286             end
287         end
288
289         try
290             close_system(modelPath,0);
291         catch
292         end
293     end
294 end
295
296 % Save all results
297 save(fullfile(saveFolder,'all_results.mat'),'results','-v7.3');
298 fprintf('\nAll sweeps finished. Results in: %s\n', saveFolder);
```

C Simulink Models

The following Simulink models were developed for load analysis simulations:

C.1 Rectifier Configuration Models

1. `center_taped.slx` - Center-tapped full-wave rectifier with thyristor control
2. `Full_Wave_Bridge.slx` - Full-wave bridge rectifier with four thyristors
3. `half_wave.slx` - Half-wave controlled rectifier with single thyristor

C.2 Diode Reference Models

1. `ct_diode.slx` - Center-tapped rectifier with diodes (uncontrolled)
2. `fw_diode.slx` - Full-wave bridge with diodes (uncontrolled)
3. `hf_diode.slx` - Half-wave rectifier with diode (uncontrolled)

All Simulink models are located in the `load_analysis/simulink/` directory and implement various load scenarios including resistive-only, R-L, and highly inductive loads. The models support automated parameter sweeps through the MATLAB script interface for comprehensive analysis of rectifier performance across different firing angles and load conditions.

C.3 Simulink/Simscape Circuit Diagrams

The following figures show the Simscape implementation of the controlled rectifier circuits used in the load analysis simulations.

C.3.1 Center-Tapped Full-Wave Rectifier

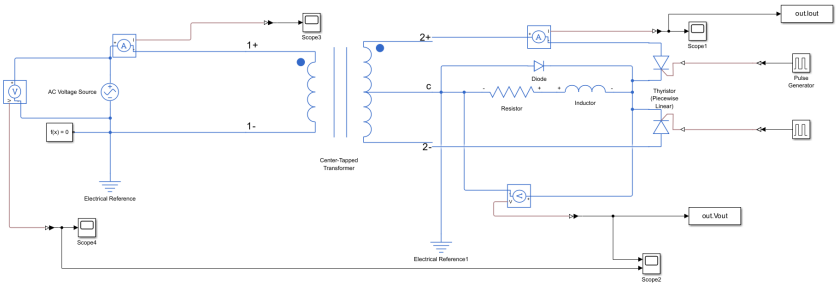

Figure 38: Center-tapped full-wave controlled rectifier Simscape model. The circuit uses two thyristors with phase-shifted gate signals to achieve full-wave rectification. A center-tapped transformer provides the dual voltage sources required for this topology.

C.3.2 Full-Wave Bridge Rectifier

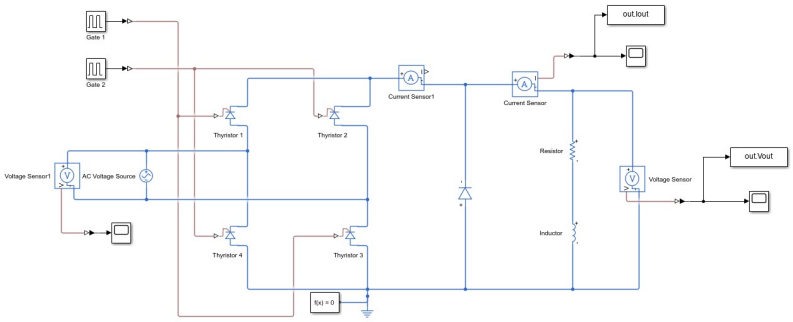

Figure 39: Full-wave bridge controlled rectifier Simscape model. The circuit employs four thyristors arranged in a bridge configuration, allowing for full-wave rectification without requiring a center-tapped transformer. Gate signals are synchronized to control the firing angles of thyristor pairs.

C.3.3 Half-Wave Rectifier

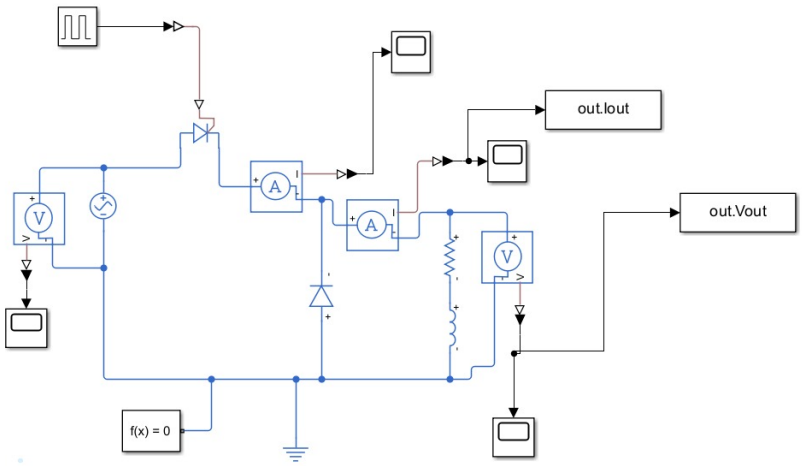

Figure 40: Half-wave controlled rectifier Simscape model. This simplified topology uses a single thyristor to control current flow during the positive half-cycle of the AC input. The circuit demonstrates the basic principles of phase-controlled rectification.